

Карпатський національний університет імені Василя Стефаника

Факультет математики та інформатики

Кафедра алгебри та геометрії

ДИПЛОМНА РОБОТА

на здобуття першого (бакалаврського) рівня вищої освіти

на тему “Векторна алгебра в контексті сучасних методів комп’ютерної графіки”

Студента IV курсу, групи М-41
спеціальності 111 “Математика”

Халімова Івана Васильовича

Керівник: к.ф.-м.н. Глушак І.Д.

Рецензент: к.ф.-м.н. Копорх К.М.

Зміст

Вступ	1
Розділ 1. Основи векторної алгебри та її застосування в комп'ютерній графіці	3
1.1. Поняття вектора та його координатне подання у тривимірному просторі.	3
1.2. Лінійні операції над векторами: додавання, віднімання та множення на скаляр.	4
1.3. Нормалізація векторів.	6
1.4. Скалярний добуток векторів і його застосування.	7
1.5. Векторний добуток та його застосування.	9
1.6. Мішаний добуток та його застосування.	11
1.7. Проекції векторів та їх використання в комп'ютерній графіці.	12
Розділ 2. Розробка 3D тренажера для роботи з векторами	15
2.1. Постановка задачі та вимоги до програмного застосунку.	15
2.2. Загальна структура.	16
2.3. Реалізація математичної моделі вектора	19
2.4. Реалізація операцій над векторами.	24
2.5. Візуалізація сцени за допомогою графічного API OpenGL.	33
2.6. Візуалізація операцій з векторами	41
2.7. Реалізація користувацького інтерфейсу та керування камерою.	46
2.8. Перевірка коректності роботи та демонстраційні приклади	49
Висновки	62
Список використаних джерел	63

Вступ

Сучасні засоби комп'ютерної графіки надають можливість створювати складні тривимірні сцени, реалізовувати моделі освітлення та налаштувати його під певні умови, здійснювати керування параметрами камери, створювати просторові об'єкти довільної форми та моделювати динаміку їхнього руху. Формалізований опис просторових характеристик сцен та геометричних об'єктів в ній забезпечує сукупність методів лінійної алгебри, аналітичної та обчислювальної геометрії, які базуються на векторній алгебрі.

Методи векторної алгебри застосовуються для опису геометричних об'єктів, позиціонування на визначення напрямків руху об'єктів, визначення нормалей до площин та поверхонь, реалізації геометричних перетворень, визначення параметрів освітлення та віртуальної камери.

Актуальність теми дослідження зумовлена тим, що для розуміння сучасних технологій тривимірної візуалізації та графічного програмування, необхідне вміння поєднання формального аналітичного апарату векторної алгебри з геометричною інтерпретацією в тривимірному просторі. Запис математичних формул та отримані з них числові характеристики не завжди дають чітке усвідомлення їхнього геометричного змісту. Тому доцільним є створення інтерактивного графічного додатку, який здійснює реалізацію операцій над векторами у тривимірному просторі та наочне дослідження їх властивостей.

Метою дипломної роботи є аналіз методів векторної алгебри в контексті сучасної комп'ютерної графіки та розробка програмного застосунку для інтерактивної візуалізації операцій над векторами у тривимірному просторі. Для досягнення поставленої мети необхідно виконати такі завдання:

- Систематизувати базові поняття векторної алгебри;
- Визначити, як математичні операції над векторами застосовуються у комп'ютерній графіці;
- Описати структуру подання вектора для подальшої програмної реалізації;
- Реалізувати програмне виконання операцій з векторами;
- Створити тривимірну сцену для візуального відображення векторів та операцій над ними;
- Реалізувати побудову проекцій векторів, кутів, площин та інших допоміжних елементів;
- Забезпечити для користувача зручне користування інтерфейсом та управління віртуальною камерою;
- Перевірити коректність відображення результатів виконання операцій над векторами для демонстраційних прикладів.

Об'єктом дослідження є математичні моделі опису та геометричного моделювання об'єктів тривимірного простору, які застосовуються у комп'ютерній графіці.

Предметом дослідження є методи векторної алгебри та їх застосування для опису геометричних об'єктів, просторових геометричних перетворень та проектування, для моделювання освітлення та візуалізації тривимірних сцен.

У роботі використовуються методи лінійної алгебри, аналітичної геометрії, об'єктно-орієнтованого програмування та засоби комп'ютерної візуалізації. Програмний застосунок реалізовано мовою C++ із використанням графічного API OpenGL та його бібліотеки GLUT.

Дипломна робота складається зі вступу, двох розділів, висновків та списку використаних джерел. У першому розділі розглядається теоретичні основи векторної алгебри та описуються її основні застосування в комп'ютерній графіці. У другому розділі описано структуру та реалізацію програмного застосунку, проведено тестування його роботи.

Розділ 1. Основи векторної алгебри та її застосування в комп'ютерній графіці

1.1. Поняття вектора та його координатне подання у тривимірному просторі.

Означення 1. Вектором називається напрямлений відрізок, для якого важливими є дві характеристики: напрям і довжина. Вектор прийнято позначати $\vec{a}$ або $\overrightarrow{AB}$.

У комп'ютерній графіці вектори є основою для представлення моделей просторових об'єктів та опису переміщення, задання напрямку руху чи орієнтації.

Задається вектор $\vec{a}$ координатами точок, що є його кінцями, зазвичай відносно прямокутної декартової системи координат. Для довільного n -вимірного простору $\mathbb{R}^n$ в координатній формі вектор представляється у вигляді:

$$\vec{a} = \begin{pmatrix} a_x \\ a_y \\ a_z \\ \vdots \\ a_t \end{pmatrix} = \begin{pmatrix} x_2 - x_1 \\ y_2 - y_1 \\ z_2 - z_1 \\ \vdots \\ t_2 - t_1 \end{pmatrix},$$

де $(a_x, a_y, a_z, \dots, a_t)$ - це координати вектора із кінцем у точці з координатами $(x_2, y_2, z_2, \dots, t_2)$ та початком в точці $(x_1, y_1, z_1, \dots, t_1)$. Оскільки ми розглядаємо простір $\mathbb{R}^3$ то обмежимося розглядом тільки векторів $\vec{a} = (a_x, a_y, a_z)$.

Геометрично вектор зображається у вигляді напрямленого відрізка. Отже, координати вектора показують, на скільки одиниць повинна зміститися точка в напрямку кожної із відповідних координатних осей, при русі вздовж вектора від початкової точки до кінцевої. Якщо ж початок вектора збігається з центром системи координат, тобто $O(0,0,0)$, то такий вектор задається тільки координатами свого кінця, тобто:

$$\vec{a} = (x_2 - 0, y_2 - 0, z_2 - 0) \Rightarrow \vec{a} = (x_2, y_2, z_2)$$

Важливо розрізняти між собою координати точки та вектора. Точка тільки визначає положення у просторі, а вектор задає напрям. В комп'ютерній графіці ця відмінність має суттєве значення, оскільки одні й ті ж числові координати матимуть різний зміст в залежності від умови: опис положення, чи задання напрямку руху об'єкта в просторі.

Означення 2. Довжиною (модулем) вектора називають довжину відрізка, яким він зображається, тобто відстань між початком та кінцем даного вектора. Для знаходження довжини вектора використовують таку формулу:

$$|\vec{a}| = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

Означення 3. *Нульовим вектором* називають вектор з координатами $(0,0,0)$ і довжиною, яка дорівнює 0. Тобто це вектор, в якого кінець та початок співпадають. Слід зауважити, що деякі операції над векторами, вимагають попередньої перевірки того, чи вектор є відмінним від нульового, оскільки, наприклад, нормалізація вектора або визначення кута між векторами, не мають коректного змісту для вектора з нульовою довжиною.

Нульовий вектор є особливим, оскільки він не має ні напрямку, ні довжини, а при його додаванні до іншого вектора він його не змінює (тобто $\vec{0} + \vec{a} = \vec{a}$.)

Координатне задання вектора в комп'ютерній графіці є основою для моделювання тривимірного віртуального простору та програмної реалізації операцій векторної алгебри, що дозволяє описувати напрямки, та інші перетворення в цьому просторі. Слід зауважити, що у межах розробленого програмного застосунку тривимірний вектор подається як структура, що задається трьома координатами. Для візуального відображення вектора у вигляді стрілки зберігаються координати його початкової та кінцевої точок, координати вектора обчислюється як різниця цих відповідних координат.

1.2. Лінійні операції над векторами: додавання, віднімання та множення на скаляр.

До базових операції векторної алгебри належать додавання та віднімання векторів, а також множення вектора на скаляр. Ці операції є основою для складніших обчислень, які використовуються безпосередньо в комп'ютерній графіці для побудови сцени, моделювання руху та інших дій.

Означення 4. *Сумою векторів $\vec{a}$ та $\vec{b}$* називають вектор проведений з початку вектора $\vec{a}$ до кінця вектора $\vec{b}$ при умові, що кінець вектора $\vec{a}$ співпадає з початком вектора $\vec{b}$. Їх суму позначають $\vec{a} + \vec{b}$ та визначають в координатній формі так:

$$\vec{a} + \vec{b} = (a_x + b_x, a_y + b_y, a_z + b_z)$$

Геометрично додавання векторів можна пояснити за допомогою певних правил: **“правило трикутника”** та **“правило паралелограма”**. За *правилом трикутника* початок другого вектора ($\vec{b}$) переносять у кінець першого вектора ($\vec{a}$), а результат зображають вектором, початок якого є початок вектора $\vec{a}$, а кінець - кінець другого вектора $\vec{b}$, таким чином утворюється трикутник. *Правилом паралелограма* застосовується до векторів $\vec{a}$ та $\vec{b}$ що мають спільний початок, після чого добудовують додаткові вектори ($\vec{a}$ та $\vec{b}$) таким чином, що початок вектора $\vec{b}$ збігається з кінцем вектора $\vec{a}$, а $\vec{a}$ - з вектором $\vec{b}$. Внаслідок утворюється паралелограм, а результат додавання векторів – це його діагональ, яку зображають у вигляді вектора, для якого початок – спільний початок $\vec{a}$ та $\vec{b}$, а кінець – спільний кінець $\vec{a}$ та $\vec{b}$.

В комп'ютерній графіці додавання векторів часто використовується для опису переміщення геометричного об'єкта, визначеного координатами своїх вершин, зокрема при реалізації анімації руху об'єкта, моделюванні руху камери та інших дій в просторі сцени.

Означення 5. *Різницею векторів $\vec{a}$ та $\vec{b}$* називають вектор $\vec{t}$ такий що, $\vec{b} + \vec{t} = \vec{a}$. Різницю позначають $\vec{a} - \vec{b}$ та в координатній формі визначається так:

$$\vec{a} - \vec{b} = (a_x - b_x, a_y - b_y, a_z - b_z)$$

Різниця векторів має властивість: різницю $\vec{a}$ та $\vec{b}$ можна отримати, додаючи вектори $\vec{a}$ та $-\vec{b}$, тобто для різниці виконується така рівність:

$$\vec{a} - \vec{b} = \vec{a} + (-\vec{b})$$

яка не впливає на алгебру, але відображається по різному з геометричного точки сенсу.

У задачах комп'ютерної графіки віднімання векторів має важливе практичне значення, зокрема для визначенні напрямку руху від одного до іншого об'єкта, визначати напрямку погляду віртуальної камери. Наприклад, якщо камера розташована на у певній точці C , а об'єкт знаходиться в положенні, що визначається точкою T , то напрям погляду камери може бути заданий вектором:

$$\vec{v} = T - C$$

Означення 6. *Добутком вектора $\vec{a}$ на число k* називають такий вектор $\overrightarrow{ka}$, який задовольняє такі умови:

1) Довжина $|\overrightarrow{ka}| = k \cdot |\vec{a}|$, тому для представлення результату множення вектора на скаляр в координатній формі, застосовується формула:

$$\overrightarrow{ka} = (ka_x, ka_y, ka_z)$$

2) $\overrightarrow{ka} \uparrow \vec{a}$, якщо $k > 0$ та $\overrightarrow{ka} \downarrow \vec{a}$, якщо $k < 0$.

В геометричному сенсі множення на скаляр інтерпретується як масштабування вектора:

- якщо $k > 1$ – видовжується;
- якщо $0 < k < 1$, то його довжина зменшується;
- якщо $k = 1$, то вектор не змінюється;
- якщо $k = 0$, то вектор стає нульовим;
- якщо $k < 0$, то вектор змінює довжину, та стає протилежно напрямленим.

У комп'ютерній графіці множення вектора на скаляр k використовується для масштабування, обчислень зміщень, зміни розмірів об'єктів, керування швидкістю

руху при анімації тощо. Наприклад, якщо напрям руху об'єкта задано одиничним вектором, а швидкість задається певним значенням, тоді переміщення за певний проміжок часу буде визначатися значенням k . Цей принцип використовується в анімації, фізичних симуляціях та системах інтерактивної графіки.

У межах роботи створено інтерактивне графічне середовище, яке візуалізує у наочній формі перераховані вище лінійні операції з векторами у тривимірній сцені. Додавання векторів відображається за *правилом трикутника або правилом паралелограма*, віднімання векторів показує різницю між напрямками і також використовує *правило трикутника*, а множення на скаляр демонструє зміну напрямку вибраного вектора та його довжину. Розроблено зручний інтерфейс, за допомогою якого, користувач може побачити не тільки числовий результат виконання, а й геометричний зміст даних операцій.

1.3. Нормалізація векторів.

У деяких алгоритмах, які застосовуються в комп'ютерній графіці використовуються виключно вектори, які мають довжину 1, тому виникає необхідність виконувати попереднє нормування векторів.

Означення 7. Якщо довжина вектора дорівнює одиниці, то його називають *одиничним (або ортом)*.

Одиничний вектор зберігає напрям початкового вектора. Такий вектор доволі зручно використовувати в випадках, коли потрібно відокремити напрям від величини. Наприклад, напрям руху об'єкта може бути одиничним вектором, але швидкість переміщення задається числовими коефіцієнтом. Процес перетворення довільного ненульового вектора на одиничний називається нормалізацією.

Означення 8. Нормалізацією вектора вважаються операцію, що перетворює довільний вектор на одиничний, тобто вектор з довжиною, що дорівнює одиниці. Для перетворення довільного вектора у одиничний використовують такий спосіб:

якщо задано вектор:

$$\vec{a} = (a_x, a_y, a_z)$$

то його нормалізований вектор має вигляд

$$\vec{f} = \frac{\vec{a}}{|\vec{a}|} = \left(\frac{a_x}{|\vec{a}|}, \frac{a_y}{|\vec{a}|}, \frac{a_z}{|\vec{a}|} \right)$$

де $|\vec{a}|$ - довжина вектора $\vec{a}$.

Слід зауважити, що нормалізація можлива лише для ненульових векторів.

Нормалізація є однією з найпоширеніших операцій у комп'ютерній графіці. Вона використовується під час роботи з напрямками камери, нормальми до поверхні,

напрямками освітлення, векторами руху та іншими об'єктами. Наприклад, якщо потрібно у певному напрямку на задану відстань перемістити об'єкт, тоді напрям нормалізують, і після цього застосовують його множення на скаляр, що задається певним змінним значенням, що дозволяє об'єкту рухатися. Такий підхід сприяє уникнення неприємних ситуацій, коли швидкість або відстань залежить безпосередньо від початкової довжини вектора. Якщо ж вектор не нормалізувати, то довший вектор спричинить більше зміщення, навіть якщо логічно потрібно було використати напрямок цього ж вектора.

Особливу роль відіграють одиничні вектори при налаштуванні освітлення сцени. У багатьох моделях освітлення застосовується кут між нормаллю до поверхні та променем від джерела світла. Для правильного і коректного результату обидва ці вектора повинні бути обов'язково нормалізованими. Також нормалізація використовується під час роботи з камерою. Напрямок погляду вектора можна подати як вектор від положення камери до точки, на яку ця камера спрямована. Щоб використати цей напрям, його потрібно привести до вектора одиничної довжини і після цього окремо керувати напрямом погляду.

У розробленому програмному застосунку визначення довжина та нормалізація вектора використовується як основа для інших операцій: визначення кута, побудови одиничних напрямків, обчислення проєкцій, керування камери, тощо. Тобто ці операції не виділено окремо, але вони використовуються в усіх основних алгоритмах візуалізації та реалізації операцій над векторами.

1.4. Скалярний добуток векторів і його застосування.

Скалярний добуток є однією з основних операцій над векторами. Він має важливе значення в задачах комп'ютерної графіки. На відмінну від операцій, які було розглянуто вище, результатом скалярного добутку є число, а не вектор.

Означення 9. Скалярним добутком (позначається $\vec{a} \cdot \vec{b}$ або $\vec{a}\vec{b}$) векторів $\vec{a}$ та $\vec{b}$ називають число:

$$\vec{a} \cdot \vec{b} = |\vec{a}| \cdot |\vec{b}| \cdot \cos \alpha$$

Для скалярного добутку двох векторів $\vec{a}$ та $\vec{b}$ використовують їх довжину і множать на $\cos \alpha$, де α кут між променями побудованими вздовж векторів $\vec{a}$ та $\vec{b}$ з точки, що їхнім спільним початком, причому кут $\alpha \in [0, \pi]$. Крім цього вектори можуть бути **ортогональними**.

Означення 10. Два вектори які утворюються кут $\alpha = \frac{\pi}{2}$ називають **ортогональним**, їх позначають $\vec{a} \perp \vec{b}$.

Скалярний добуток дає змогу визначити взаємне розташування двох напрямків у просторі, зокрема кут між ними та перевірити на ортогональність. Для обчислення кута використовують таку формулу:

$$\cos \alpha = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| \cdot |\vec{b}|}$$

Тоді як сам кут визначається як:

$$\alpha = \arccos\left(\frac{\vec{a} \cdot \vec{b}}{|\vec{a}| \cdot |\vec{b}|}\right)$$

Варто зауважити, що дана формула використовується лише для ненульових векторів, оскільки у знаменнику містяться їхні довжини, а якщо хоча б одна із довжин вектора буде дорівнювати 0, тоді буде невизначеність. Для цього потрібно вводити додаткову перевірку, щоби уникнути ділення на нуль.

Основні властивості скалярного добутку:

- якщо $\vec{a} \cdot \vec{b} > 0$, то кут між векторами гострий;
- якщо $\vec{a} \cdot \vec{b} < 0$, то кут між векторами тупий;
- якщо $\vec{a} \cdot \vec{b} = 0$, то кут між векторами прямий, а отже вектори ортогональні.

Саме умова нульового скалярного добутку є найзручнішим способом для перевірки на ортогональності двох векторів.

Для одиничних векторів формула скалярного добутку спрощується, оскільки їхні довжини рівні одиниці. Це дуже зручно, адже результат одразу показує косинус кута між двома напрямками. Значення скалярного добутку (u), яке близьке до 1, означає, що вектори спрямовані майже однакою. Якщо $u = -1$ означає, що вектори протилежно напрямлені. Якщо $u = 0$ то кут – прямий.

У комп'ютерній графіці скалярний добуток має широке практичне застосування. Одним із найпоширеніших прикладів є визначення кута між напрямками. Наприклад, у тривимірному просторі може виникати потреба визначити, наскільки напрям погляду камери збігається з напрямком споглядання певного об'єкта. Або застосовується при моделюванні освітлення поверхні, оскільки в багатьох моделях інтенсивність світла залежить від кута між нормаллю поверхні та напрямком променя від джерела світла. Якщо вони спрямовані однакою, тоді освітлення сильне, якщо кут має велику різницю між нормаллю поверхні та напрямком джерела світла, тоді освітлення слабке. Така залежність вимірюється через скалярного добутку відповідних нормалізованих векторів.

Також скалярний добуток також використовується під час побудови проєкцій, визначення видимих частин об'єкта, аналізу руху об'єкта тощо. Наприклад, потрібно з'ясувати, чи рухається об'єкт у напрямку певної осі, тоді можна порівняти напрям

руху з одиничним вектором цієї осі, і якщо аналіз покаже додатній результат, тоді рух у спільному напрямку, інакше – у протилежному.

У представленому в роботі програмному додатку скалярний добуток використовується для декількох режимів роботи: визначається числове значення скалярного добутку двох векторів, кут між ними та його тип, здійснює аналіз ортогональності. При визначенні числового значення скалярного враховуються особливості роботи з числами: для дійсних значень з плаваючою комою результат математично рівний нулю, але в програмі ми працюємо з малими числами, які можуть містити в собі 7-10 значень після коми. Тому перевірка ортогональності доцільно використовується для порівняння допустимих значень похибки. Це дозволяє користувачу побачити не тільки результат, а геометричний зміст у тривимірному просторі.

1.5. Векторний добуток та його застосування.

Також слід розглянути векторний добуток, оскільки його роль в комп'ютерній графіці не менш важлива. На відмінну від скалярного добутку двох векторів, результат векторного добутку – це новий вектор. Цей вектор має особливу властивість: він перпендикулярний до кожного з вихідних векторів. в комп'ютерній графіці часто застосовують для побудови нормалей до поверхонь, визначення орієнтації полігонів та роботи з площинами.

Означення 11. Векторний (або зовнішній) добуток векторів $\vec{a}$ та $\vec{b}$ - це такий вектор $\vec{c}$ який задовольняє такі умови:

- 1) вектор $\vec{c}$ ортогональний до кожного з векторів $\vec{a}$ та $\vec{b}$;
- 2) $|\vec{c}| = |\vec{a}| \cdot |\vec{b}| \cdot \sin\beta$, де β - кут між векторами $\vec{a}$ та $\vec{b}$;
- 3) Трійка векторів базисних векторів $\{\vec{i}, \vec{j}, \vec{k}\}$ та $\{\vec{a}, \vec{b}, \vec{c}\}$ однаково орієнтовані.

Обчислюється векторний добуток (позначають $\vec{a} \times \vec{b}$) за такою формулою:

$$\vec{a} \times \vec{b} = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix}$$

Визначивши визначник, цю формулу можна переписати таким чином:

$$\vec{a} \times \vec{b} = (a_y b_z - a_z b_y) \vec{i} - (a_x b_z - a_z b_x) \vec{j} + (a_x b_y - a_y b_x) \vec{k}$$

Напрямок векторного добутку визначається *правилом правої руки*. Якщо пальці правої руки умовно спрямувати від вектора $\vec{a}$ до вектора $\vec{b}$ через найменший кут, то великий палець покаже напрям вектора $\vec{a} \times \vec{b}$.

При зміні порядку множення векторів напрям вектора, що є результатом змінюється на протилежний. Цю властивість називають *антикомунікативністю векторного добутку*.

Ця властивість важлива, зокрема при застосуванні векторного добутку в комп'ютерній графіці для встановлення орієнтації нормалі до плоскої грані багатокутника, що визначає порядок обходу вершин полігон. Якщо вершини багатокутника задані в одному напрямку, нормаль спрямована в один бік, якщо змінити порядок, тоді буде протилежність порядку і напрямку.

Довжина вектора, що є векторним добутком векторів $\vec{a} \times \vec{b}$ також має свій геометричний зміст – вона дорівнює площі паралелограма, побудованого між вихідними двома векторами. При цьому можливі часткові випадки:

- якщо вектори перпендикулярні, тоді $\sin \beta = 1$, і площа дорівнює добутку їх довжин;
- якщо ж колінеарні, тоді $\sin \beta = 0$, і векторний добуток дорівнює нульовому вектору.

Векторний добуток також можна застосовувати для перевірки вектори на колінеарність. Але варто врахувати що, якщо результатом є нульовий вектор, тоді початкові вектори можуть бути на одній прямій або один із них має нульову довжину.

Одним із головних застосувань векторного добутку – це визначення нормалі до площини або поверхні.

Означення 12. Нормаллю - називають вектор, який перпендикулярний до поверхні або до площини.

Одинична нормаль використовується в алгоритмах для освітлення, оскільки для коректного визначення яскравості розподілу світла на поверхні та інших властивостей світла в просторі, важливо саме напрям нормалі, а не її довжина. В графічних системах крім освітлення, напрям нормалі важливий для алгоритмів відсікання невидимих граней моделі. У комп'ютерній графіці часто немає потреби малювати ті грані об'єкта, які на потрапляють в область видимості. Щоб визначити видимість грані, можна порівняти її нормалі з напрямком погляду камери. Векторний добуток також використовується для роботи з локальними системами координат. Наприклад, якщо відомо напрям руху об'єкта та один допоміжний напрям, через який можна визначити третій взаємний перпендикулярний напрям. Це зазвичай використовують при зміні орієнтації камери, побудові базисів у просторі та інше.

У реалізованому в роботі програмному додатку векторний добуток використовується для візуалізації нормалі до площини, побудованій за двома вибраними векторами. Користувач може задати два вектора, після чого програма обчислює їх векторний добуток і візуалізує новоутворений вектор, який перпендикулярний до двох заданих. Також визначається його довжина, яка визначає

площу паралелограма, який для наочності також зображається. Також дана програма аналізує певні випадки, якщо два вектори колінеарні або один із цих векторів має нульову довжину. У такому випадку побудувати нормаль до площини не можливо, оскільки площина не визначена.

1.6. Мішаний добуток та його застосування.

Мішаний добуток є операцією, яка поєднує скалярний та векторний добуток. Результатом даного добутку є число, але його геометричний зміст дозволяє визначити об'єм просторової фігури використовуючи орієнтацію трьох векторів та їх взаємне розміщення.

Означення 12. Мішаним добутком векторів $\vec{a}$, $\vec{b}$ та $\vec{c}$ називається скалярний добуток вектора $\vec{a}$ на вектор, який є векторним добутком векторів $\vec{b}$ та $\vec{c}$. Згідно з цим означення даний добуток позначають так:

$$(\vec{a}\vec{b}\vec{c}) \Leftrightarrow \vec{a} \cdot (\vec{b} \times \vec{c})$$

Для обчислення мішаного добутку, спочатку обчислюють векторний добуток $\vec{b} \times \vec{c}$, результатом якого є вектор, який є перпендикулярним до площини, утвореної векторами $\vec{b}$ та $\vec{c}$. Після цього обчислюється скалярний добуток отриманого вектора $\vec{a}$, і в результаті отримуємо числове значення, яке можна записати у вигляді визначника:

$$(\vec{a}\vec{b}\vec{c}) = \begin{vmatrix} a_x & a_y & a_z \\ b_x & b_y & b_z \\ c_x & c_y & c_z \end{vmatrix}$$

де в кожному рядку послідовно записані координати відповідних векторів.

Властивості мішаного добутку:

1) циклічна перестановка не змінює величини мішаного добутку, тобто виконується рівність: $(\vec{a}\vec{b}\vec{c}) = (\vec{b}\vec{c}\vec{a}) = (\vec{c}\vec{a}\vec{b})$;

2) $((t\vec{a})\vec{b}\vec{c}) = (\vec{a}(t\vec{b})\vec{c}) = (\vec{a}\vec{b}(t\vec{c})) = t(\vec{a}\vec{b}\vec{c})$, де t - довільне число;

3) $((\vec{a}_1 + \vec{a}_2)\vec{b}\vec{c}) = (\vec{a}_1\vec{b}\vec{c}) + (\vec{a}_2\vec{b}\vec{c})$.

Якщо три вектора відкласти від спільного початку, тоді вони утворюють ребра просторового паралелепіпеда. Значення мішаного добутку показує, який об'єм займає дана фігура. Тобто при $\vec{a} \cdot (\vec{b} \times \vec{c})$ ми отримуємо число, яке і є значення об'єма фігури..

Значення мішаного добутку дає можливість встановити наступне :

- якщо знак мішаного добутку більше нуля, тоді трійка векторів має однакову орієнтацію;
- якщо знак мішаного добутку менше нуля, тоді трійка векторів має протилежну орієнтацію;

- якщо знак мішаного добутку рівний нулю, тоді об'єм паралелепіпеда дорівнює також нулю. Це дає зрозуміти, що три вектори лежать в одній площині або один із них є нульовим, у такому випадку вони не будуть утворювати тривимірну фігуру.

Мішаний добуток використовується для аналізу компланарності трьох векторів. Ця властивість є корисною, наприклад, у задачах коли потрібно визначити, чи належить певні прямі одній площині.

У комп'ютерній графіці мішаний добуток може використовуватися для аналізу просторової орієнтації, побудови локальних систем координати та інших дій. Також мішаний добуток застосовується для роботи з об'ємними моделями. Наприклад, якщо потрібно визначити, чи має задана фігура ненульовий об'єм, і якщо результат рівний нулю, тоді фігура не займає об'єму в просторі. Ще не менш важливим застосування є визначення орієнтації у просторі. Зазвичай він використовується саме для задач, де потрібно визначити, з якого боку площини розміщена точка або напрямлений вектор, а також визначити відхилення від площі і його знак.

У розробленому програмному додатку демонструється застосування мішаного добутку $(\vec{a}\vec{b}\vec{c})$ для визначення об'єму паралелепіпеда, побудованого на трьох векторах, який також візуалізується. Також наочно зображається вектор $\vec{b} \times \vec{c}$, який є нормаллю до відповідної площини, а і наскільки наступне скалярне множення на вектор $\vec{a}$ змінює значення для напрямку цієї нормалі. Якщо ж такого значення не має або не існує, тоді об'єм рівний нулю.

1.7. Проекції векторів та їх використання в комп'ютерній графіці.

Також для вектора є важливе саме поняття проєкції, яке дозволяє визначити, яку частину одного вектора можна розглядати як спрямовану вздовж іншого вектора або певної осі.

Означення 13. Геометричною проєкцією вектора на інший вектор називають вектор, який можна отримати, якщо провести перпендикуляри від кінців вектора до променя, що містить інший вектор.

Тобто проєкція початку спроектованого вектора відповідає початку геометричної проєкції, а проєкція кінця вектора - кінцю геометричної проєкції.

Означення 14. Скалярною проєкцією вектора на вісь називають скалярну величину (число), яке дорівнює довжині *геометричної проєкції вектора* на цю вісь, у випадку якщо напрями осі і геометричної проєкції збігаються, або число протилежним за знаком довжині геометричної проєкції вектора, якщо напрям осі і геометричної проєкції вектора є протилежними.

Отже, розрізняють проєкцію векторну та скалярну. Скалярна проєкція показує числову величину складової одного вектора вздовж іншого (зазвичай для осей). Вона може бути додатною, від'ємною або дорівнювати нулю. Формула має ось такий вигляд:

$$proj_{\vec{a}}\vec{b} = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}|}$$

Дана формула показує, наскільки вектор $\vec{b}$ спрямований уздовж вектора $\vec{a}$:

- якщо результат більше нуля, то складова вектора $\vec{b}$ має той самий загальний напрям, що й вектор $\vec{a}$;
- якщо результат менше нуля, то складова вектора $\vec{b}$ має протилежний напрям;
- якщо результат дорівнює нулю, то вектор $\vec{b}$ перпендикулярний до вектора $\vec{a}$.

На відмінну від скалярної проєкції, векторна проєкція подається вектором. Вона має напрям вектора $\vec{a}$ і довжину, що відповідає складовій вектора $\vec{b}$ вздовж цього напрямку. Сама ж формула теж відрізняється і має ось такий вигляд:

$$proj_{\vec{a}}\vec{b} = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}|^2} \vec{a}$$

Якщо вектор $\vec{a}$ є одиничним (тобто довжина рівна одиниці), то формула спрощується:

$$proj_{\vec{a}}\vec{b} = (\vec{a} \cdot \vec{b}) \vec{a}$$

В багатьох графічних обчисленнях для напрямків, на який використовується проєктування, використовується нормалізація, що забезпечує зручність для реалізацій.

Простими словами, у геометричному розумінні, проєкція відображає “тінь” одного вектора на інший. Це поняття використовується в комп'ютерній графіці, зокрема для побудови тіней від об'єкта на площину. Також застосовується під час розкладання руху на складові. Наприклад, потрібно визначити проєкцію вектора $\vec{b}$ на напрям вектора $\vec{a}$. В такому випадку вектор $\vec{a}$ задає напрям, на який виконується проєктування, а вектор $\vec{b}$ є тим вектором, який розкладає на складові. Один із таких прикладів є рух об'єкта вздовж певної поверхні. Якщо об'єкт має напрям руху, але певна частина його руху спрямовано перпендикулярно до площини, тоді можна відокремити складову, яка лежить у площині, що дозволяє використовувати її. Зазвичай його використовують для фізичних симуляцій, навігації по поверхнях, тощо. Також проєкція є основою для механіки руху камери, наприклад, для розміщення камери відносно локальних напрямків. Для цього потрібно знати складові руху вздовж напрямку погляду. Також проєкція застосовується для обчислення відстані від точки до площини, якщо відома нормаль площини, тоді вектор від точки площини до певної довільної точки можна спроектувати на цю нормаль, де довжина такої проєкції, показує відстань між віддаленою точкою та площиною, яка перпендикулярна в

напрямку. Ця можливість в комп'ютерній графіці використовується для освітлення, а саме:

- якщо напрям світлового променя сильно проєктується на нормаль, тоді поверхня освітлюється інтенсивніше;
- якщо проєкція мала або від'ємна, це означає, що світло падає під великим кутом або поверхня розвернута від джерела світла.

У даному програмному додатку демонструється проєкція для наочного показу розкладання одного вектора на інший. Крім самої проєкції, в наявності є перпендикулярна складова, що дозволяє краще зрозуміти її геометричний зміст в просторі. При цьому слід враховувати випадок, що коли вектор має нульову довжину, тоді вектор, на який виконується проєктування, є нульовим, значить формули не мають змісту.

Розділ 2. Розробка 3D тренажера для роботи з векторами

2.1. Постановка задачі та вимоги до програмного застосунку.

Однією з головних завдань даної дипломної роботи полягає у розробці програмного забезпечення для інтерактивної візуалізації операцій векторної алгебри у тривимірному просторі. Програма повинна об'єднати математичні обчислення з графічним поданням результатів, щоби користувач міг побачити як числове значення певних операцій, так і його відповідно геометричне відображення. Також дана програма є інструментом, в якого основне призначення - демонстрація операцій над векторами в навчальному контексті. Тому увага акцентуватиметься саме на коректному поданні математичного змісту, зручності управління сценою, читабельним інтерфейсом, а також на легкій та зручній формі введення даних для подальшого опрацювання.

Користувач повинен мати можливість задавати вектори у тривимірному просторі, обирати потрібно операцію і в залежності від операції вибрати пару векторів, над яким він буде працювати. Також отримати результат у вигляді числа або вектора. Дані вектори відображаються у вигляді напрямлених відрізків зі стрілками та візуальною точкою, які демонструють зрозумілий початок та кінець вектора. Для певних операцій також використовуються допоміжні вектори, зображення площин, дуг кутів, проєкцій, тощо.

Крім цих вимог по функціоналу, є також нефункціональні вимоги, в якому програма має бути простим для використання в режимі реальному часі. До нефункціональних вимог відносяться:

- коректність математичних обчислень;
- стабільна робота при введення різних координат;
- перевірку некоректних випадків;
- зрозумілий інтерфейс користувача (HUD);
- наочність графічного подання в 3D сцені;
- можливість керування камерою та HUD, а також її альтернативність;
- модульну структуру програмного коду.

Таким чином, у межах програмного застосунку, крім обробки даних та її функціоналу в використанні математичних операцій над вектором, необхідно створити перевірку, яка в залежності від випадків буде видавати той чи інший результат, а також задання зручності для користувача і оптимізація від наявних проблем програмного коду.

2.2. Загальна структура.

Даний проєкт має модульну структуру, у якій окремі частини відповідають за математичні обчислення, збереження стану сцени, обробку введення, керування камерою, виведення інформації та відображення певних операцій на координатній осі для зрозумілої для користувача. Такий підхід дозволяє розділити логіку проєкта на незалежні компоненти та уникнути ситуації, коли компоненти графіки та інтерфейсу змішуються в одному файлі з математичними операціями над векторами, що дає кращу читабельність.

Основною ідеєю структури програми є поділ на три рівні: модель даних, логіка обчислення та відображення на сцені певних векторів та їх операцій. Модель даних зберігає інформацію про вектори, поточний режим роботи, результати операцій та стан інтерфейсу. Логіка обчислення відповідає за виконання математичних операцій над векторами. Візуальна частина будує тривимірну сцену, координатні осі, вектори з наконечною стрілкою та точкою, яка показує початок цього ж вектора, а також площини, дуги кутів, інформаційної панелі користувача (HUD).

Центральним елементом програми є модель сцени, яка зберігає всі дані. До даної сцени входять вектори введені користувачем, результати обчислення, поточна операція, активні пари векторів, параметри інтерфейсу та стан введення.

Основні модулі програмного проєкту наведено нижче в таблиці 1:

Модуль	Призначення
SceneTypes	Опис базових структур: вектори, поверхні, кольори векторів та інших елементів візуального відображення (кути, проєкції, тощо), режими роботи
VectorMath	Реалізація математичних операцій векторної алгебри
OperationEngine	Обчислення результатів вибраної операції та підготовка даних для відображень результатів певної операції на сцені
SceneRender	Відображення тривимірної сцени, координатних осей та інше
HudRender	Виведення результатів інформаційної панелі, формул операцій, їх результатів, а також підказок
InputController	Обробка клавіатури для математичних операцій та керування камери, альтернативні керування без миші, введення координат, а також пари векторів для певних операцій
CameraController	Керування положенням і напрямком камери на сцену

SceneController	Координація змін у сцені, вибору операцій та оновлення результатів
PresetLibrary	Збереження готових прикладів для демонстрації
GlutApp	Ініціалізація API OpenGL та його бібліотеки GLUT, головний цикл програми та системні події

Таблиця 1. Основні модулі програмного забезпечення.

Математична частина програми винесена в окремий модуль, оскільки це важливо для операцій і вони не повинні залежати від графічної бібліотеки або їх способу відображення. Це дозволяє спростувати перевірку правильності обчислень і дозволяє використовувати одні й ті самі математичні функції для побудови результатів векторів певних операцій та для тестування.

Також для обчислення вибраної операції також існує окремий модуль, який значно спрощує пошук аналізу поточного режиму. Даний модуль також бере потрібні вектори зі сцени, виконує їх обчислення та формує набір графічних примітивів для відображення. Завдяки цього рендеринг не виконує математичні обчислення, а лише відображає вже готові дані.

Графічна частина також реалізована окремими компонентами для тривимірної сцени та інтерфейсу користувача. Генерація сцени відповідає за побудову елементів геометрії (координатні осі, вектор, допоміжні вектори та площини, тощо). Інформаційна панель відображає їх поточний режим, координати векторів, формули операцій, результати обчислень та підказок.

Також для введення певних даних також створений окремий модуль, який дозволяє користувачу вводити координати, вибір операції, обробку певних клавіш на клавіатурі/миші. А також керування камерою також винесено в окремий компонент, що дозволяє змінювати масштаб сцени, переміщення камери до точки огляду. Дані модуль є дещо зв'язані між собою, оскільки для певних дій над камерою запрограмовано за допомогою миші, але якщо в користувача немає в наявності миші, тоді є альтернативні дії за допомогою клавіатури.

Зв'язки між модулями можна подати у вигляді наступної схеми:

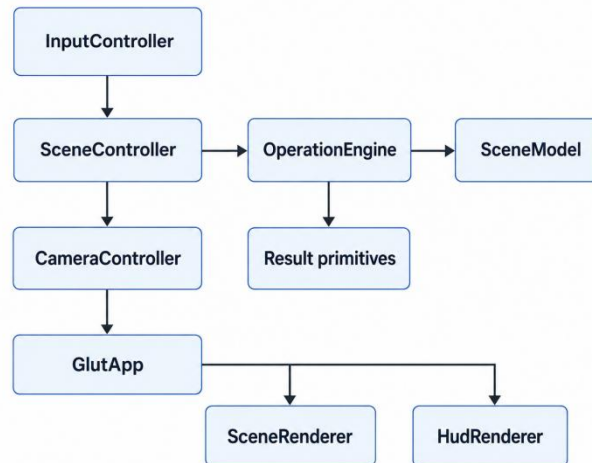


Рисунок 1. Схема зв'язку модулів.

На схемі зображено, що введення користувача обробляється контролером InputController, після чого зміни передаються до SceneController. Далі від вибраної операції використовується OperationEngine, який працює з SceneMode та формує результати для подальшого відображення. На далі компоненти SceneRender та HudRender відповідають уже за графічне подання на сцені та результатів на HUD.

Приклад ініціалізації компонентів програми виконано в наступному коді:

```

#include "CameraController.h"
#include "GlutApp.h"
#include "HudRenderer.h"
#include "InputController.h"
#include "OperationEngine.h"
#include "PresetLibrary.h"
#include "SceneController.h"
#include "SceneRenderer.h"
#include "SceneTypes.h"
#include "TextRenderer.h"
#include "VectorMathSelfTest.h"

#include <string>
void showMessage(const std::string&message, double seconds);
SceneModel scene;
CameraController camera;
OperationEngine operationEngine(scene);
TextRenderer textRenderer(scene);
SceneRenderer sceneRenderer(scene, camera, [](const Vec3&p, const
std::string&text, constColor&color) {
    textRenderer.draw3D(p, text, color);
});

constPresetLibrary& presetLibrary = PresetLibrary::defaults();
HudRenderer hudRenderer(scene, camera, presetLibrary);
SceneController sceneController(scene, camera, operationEngine, presetLibrary,
SceneController::Callbacks{

```

```

        showMessage,
GlutApp::requestRedisplay
});
InputController inputController(scene, camera, InputController::Callbacks{
    [](OperationMode mode) { sceneController.setOperation(mode); },
    [](int index) { sceneController.loadPreset(index); },
    [](int first, int second) { sceneController.selectVectorPair(first,
second); },
    []() { sceneController.clearAllVectors(); },
    []() { sceneController.rebuildOperation(); },
    []() { sceneController.resetAnimation(); },
    []() { VectorMathSelfTest::run(); },
    showMessage
});
GlutApp app(scene, camera, sceneController, sceneRenderer, hudRenderer,
textRenderer, inputController, presetLibrary);

void showMessage(const std::string&message, double seconds = 3.0) {
    scene.ui.hudMessage = message;
    scene.ui.hudMessageTimer = seconds;
}

int main(int argc, char** argv) {
return app.run(argc, argv);
}

```

У цьому фрагменті коду видно, що основні об'єкти створюються окремо, але працюють зі спільною моделлю сцени, що відповідає вимогам до кожного модуля з **таблиці 1**. Загалом структура проєкта побудована так, щоб кожний модуль мав чітко визначену відповідальність, що робить програму зручною для розширення, оскільки режими візуалізації чи нові операції над векторами можна додавати без повної зміни всієї архітектури проєкта.

2.3. Реалізація математичної моделі вектора

Математична модель векторів у програмному застосунку реалізована таким чином, щоби один і той самий тип даних можна було використовувати для обчислення та побудови графічних об'єктів у просторі. Основним цієї моделі є структура під назвою *Vec3*, яка описує вектор або точку у просторі за допомогою координати x , y , z .

У програмі координати зберігаються за значенням типу *double*, що допомагає обчислити з достатньою точністю, що є важливим фактору для опрацювання певних операцій, де визначається довжина, кут, проєкція або нормалізація. Крім самих координат структура містить базові операції векторної алгебри: додавання, віднімання, множення на скаляр, скалярний та векторний добуток, колінеарність, мішаний добуток, обчислення довжини та інше.

Основні елементами математичної моделі наведено в таблиці 2:

Структура	Призначення
Vec3	Подання точки або вектора у тривимірному просторі
Color	Збереження кольору графічного елемента
VectorSlot	Опис користувацького вектора з початковою та кінцевою точками
Arrow3D	Графічне подання вектора або результату операції у вигляді стрілки
Surface3D	Графічне подання площини або поверхні в тривимірному просторі
Arc3D	Подання дуги кута між векторами в сцені
SceneModel	Загальна модель сцени, що зберігає вектори, їх результат та поточний стан програми

Таблиця 2. Основні структури векторної математичної моделі.

З даної *таблиці 2*, можна побачити, які саме структури взаємодіють між собою і яка їх роль в даній програмі. Детальніше розглянемо нижче:

```

Struct Vec3 {
    double x = 0.0;
    double y = 0.0;
    double z = 0.0;
    Vec3() = default;
    Vec3(double px, double py, double pz) : x(px), y(py), z(pz) {}

    Vec3operator+(const Vec3&v) const { return Vec3(x + v.x, y + v.y, z +
v.z); }
    Vec3operator-(const Vec3&v) const { return Vec3(x - v.x, y - v.y, z -
v.z); }
    Vec3operator-() const { return Vec3(-x, -y, -z); }
    Vec3operator*(double k) const { return Vec3(x * k, y * k, z * k); }
    Vec3operator/(double k) const { return Vec3(x / k, y / k, z / k); }
    double dot(const Vec3&v) const { return x * v.x + y * v.y + z * v.z; }

    Vec3 cross(const Vec3&v) const {
        Return Vec3(
            y * v.z - z * v.y,
            z * v.x - x * v.z,
            x * v.y - y * v.x
        );
    }

    double length() const { return std::sqrt(dot(*this)); }

```

```

Vec3 normalized() const {
    double l = length();
    if (l < EPS) return Vec3();
    return *this/ l;
}
};

```

У даному фрагменті коду, який показано вище, видно базові математичні дії, які безпосередньо реалізовані для подальших дії вектора у просторі. Це робить код більш компактним та зрозумілим, оскільки операції над векторами можна записувати у їх звичній формі, які були описані в *Розділі 1*. Також варто звернути увагу на метод *normalized()*, оскільки перед діленням координат на довжину виконується перевірка, коли потрібно ділити число на дуже мале або на нуль.

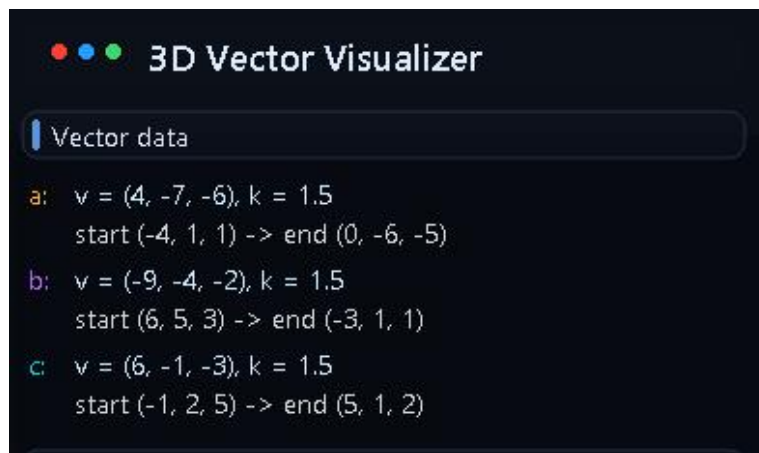


Рисунок 2. Введення початкових та кінцевих координат векторів.

Для представлення вектора, який вводить користувач, однієї структури *Vec3* не буде достатньою для повної роботи. Тому у тривимірній сцені вектор повинен обов'язково ввести дані початкових та кінцевих координат точок, також для кращої візуалізації, вектори мають свої власні кольори, назву та ознаку закінчення введення даних, що можна побачити в наступному фрагменті коду:

```

struct VectorSlot {
    std::string name;
    Vec3 start;
    Vec3 end;
    Color color;
    bool visible = true;
    bool hasStart = false;
    bool hasEnd = false;

    double scalar K = 1.50;

    bool complete() const { return visible && hasStart && hasEnd; }
    Vec3 vec() const { return complete() ? end - start : Vec3(); }
};

```

Дана структура, дозволяє не тільки зберігати інформацію координат вектора, а також його додаткову інформацію. Поля *start* та *end* описують початкову та кінцеву точки, а

метод *vec()* обчислює саме напрямлений вектор як різницю між кінцевою та початковою точками. Якщо вектор ще не введений повністю, тоді даний метод повертає нульовий вектор.

Таке застосування обґрунтовується таким чином, що користувач може вводити координати поступово, до тих міг поки не буду введені всі необхідні дані, саме тому вектор не повинен не може бути обчислений і не повинен відображатися як уже завершений об'єкт комп'ютерної графіки.

Крім користувацьких векторів, програма зберігає результати операцій. Для цього було побудовано окремі структури *Arrow3D*, *Surface3D* та *Arc3D*. Вони не виконують ніяких обчислень, але є важливими для подання результату на сцену. Загальний стан програми зберігається у класі *SceneModel*, який містить масив користувацьких векторів, списки результатів, поточний режим роботи та інше. Саме ця модель є основою для обчислення, рендера та інтерфейсу користувача:

```
class SceneModel {
public:
    std::array<VectorSlot, 3> vectors = {
VectorSlot{ "a", Vec3(), Vec3(), Color{1.00f, 0.70f, 0.16f, 1.0f}, true, false,
false },
VectorSlot{ "b", Vec3(), Vec3(), Color{0.72f, 0.38f, 1.00f, 1.0f}, true, false,
false },
VectorSlot{ "c", Vec3(), Vec3(), Color{0.00f, 0.86f, 0.82f, 1.0f}, true, false,
false }
    };

    std::vector <Arrow3D> resultArrows;
    std::vector <Surface3D> resultSurfaces;
    std::vector <Arc3D> resultArcs;
    std::vector <std::string> resultLines;

    OperationMode currentMode = OperationMode::None;
    std::string operationTitle;
    std::string formulaLine = "Enter a vector first: start point and end point.";
    std::string lessonLine = "A vector appears only after both coordinates are
entered.";
    int activePairFirst = 0;
    int activePairSecond = 1;
    int scalarVectorIndex = 0;
    int operationVectorIndex = 0;

    SceneInputState input;
    SceneUiState ui;

    void clearResults() {
        resultArrows.clear();
        resultSurfaces.clear();
        resultArcs.clear();
        resultLines.clear();
    }
};
```



```

    }
    int completeVectorCount() const {
        int count = 0;
        for (const auto& v : vectors) {
            if (v.complete()) ++count;
        }
        return count;
    }
};

```

У даної моделі сцени передбачено три основні користувацькі вектори: $\vec{a}$, $\vec{b}$ та $\vec{c}$. Цього більше чим достатньо для реалізації бінарних операцій, які потребують два вектора, або операцій які потребують три вектора (операція мішаного добутку). Також в даному фрагменті кода, можна побачити списки з такими назвами: ***resultArrows***, ***resultSurface***, ***resultArcs*** та ***resultLines***. Вони використовуються для збереження результатів поточної операції, яка на даний момент використовується. Такий спосіб надає можливість формувати результат у вигляді даних, а пізніше відображати у тривимірній сцені для певних операцій.

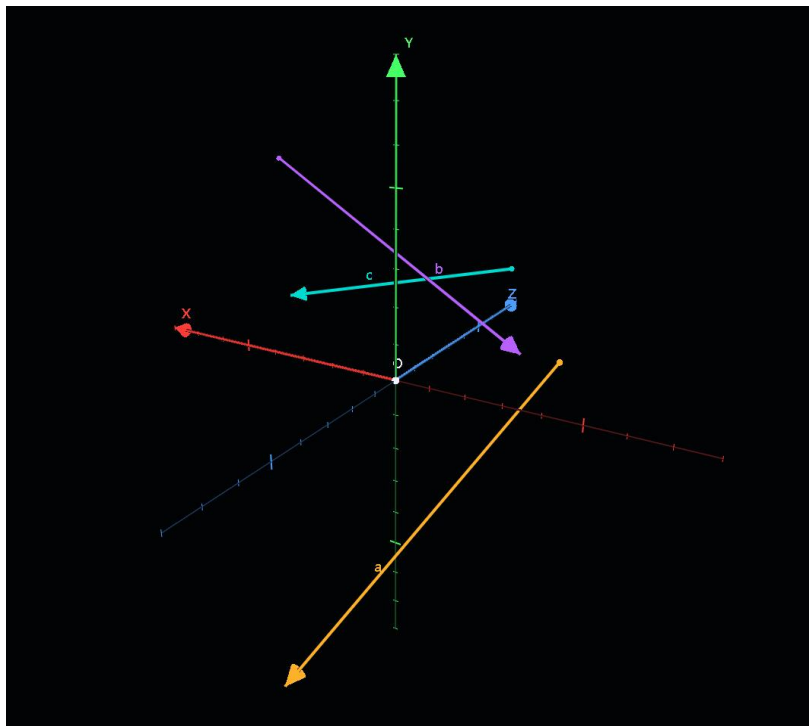


Рисунок 3. Відображення векторів в тривимірній сцені.

Таким чином, математична модель векторів у програмі побудована навколо структури ***Vec3***, яка забезпечує базові операції над векторами в просторі. А також для інтерактивної роботи використання структури ***VectorSlot***, яка зберігає вектор як пару точок і додаткові властивості для відображення.

2.4. Реалізація операцій над векторами.

Реалізація операцій над векторами побудована так, що програма спочатку визначає, яку операцію потрібно виконати, обчислює відповідні значення, формує допоміжні графічні об'єкти, а уже після цього передає результат для візуалізації у сцені для явного показу результату у вигляді відповідного результуючого вектора.

Такий підхід є основою для навчального застосунку, а також читабельним для користувача, оскільки кожна операція має не тільки числовий результат, а й геометричне подання. Наприклад: для операції додавання векторів за певним правилом будується допоміжні вектори та фігура у вигляді паралелограма або трикутника (залежно від правила).

У програмі підтримуються всі описані у першому розділі операції векторної алгебри. Їх перелік наведено нижче в таблиці:

Операція	Призначення
Довжина вектора	Обчислення модуля вибраного вектора
Додавання векторів	Побудова суми за правилом трикутника або паралелограма
Віднімання векторів	Побудова різниці двох напрямків
Множення на скаляр	Масштабування вибраного вектора
Колінеарність	Перевірка паралельності або протилежності напрямків між векторам
Кут між векторами	Обчислення та візуалізація кута між двома векторами
Скалярний добуток	Загальна модель сцени, що зберігає вектори, їх результат та поточний стан програми
Векторний добуток	Побудова вектора, побудований до двох заданих векторів
Ортогональність	Перевірка перпендикулярності між двома векторами
Нормаль до площини	Побудова одиничної нормалі та проєкції третього вектора на площину
Проекція	Розкладання одного вектора відносно іншого
Мішаний добуток	Обчислення об'єму паралелепіпеда за допомогою трьох векторів в просторі

Таблиця 3. Основні структури математичної моделі.

Перелік доступних режимів з даної *таблиці 3* задається через перелічувальний список *OperationMode*, що дозволяє зберігати поточну операцію у стані сцени та обробляти її:

```
enumclass OperationMode {
    None,
    Lengths,
    AddParallelogram,
    AddTriangle,
    Subtract,
    Scalar,
    Collinearity,
    Angle,
    Dot,
    Orthogonality,
    Projection,
    Cross,
    Mixed,
    SurfaceNormal
};
```

Окрему роль також відіграє клас *VectorMath*. Він містить базові операції векторної алгебри що не залежить від API OpenGL або інтерфейсу користувача. Завдяки цьому “математику” можна використовувати як під час побудови сцени, так і під час перевірки правильності обчислень:

```
class VectorMath {
public:
    static double length(const Vec3&v) {
        return v.length();
    }
    static double dot(const Vec3&a, const Vec3&b) {
        Return a.dot(b);
    }
    static Vec3 cross(const Vec3&a, const Vec3&b) {
        return a.cross(b);
    }
    static double mixed(const Vec3&a, const Vec3&b, const Vec3&c) {
        return dot(a, cross(b, c));
    }

    static bool angle(const Vec3&a, const Vec3&b, double &radians) {
        double lenA = length(a);
        double lenB = length(b);
        if (lenA < EPS || lenB < EPS) return false;

        double cosine = std::clamp(dot(a, b) / (lenA * lenB), -1.0, 1.0);
        radians = std::acos(cosine);
        return true;
    }

    Static Vec3 projectOntoUnitAxis(const Vec3&v, const Vec3&unitAxis) {
        return unitAxis* dot(v, unitAxis);
    }
};
```

```

    }

    static bool scalarProjection(const Vec3&v, const Vec3&axis, double &scalar) {
        double axisLength = length(axis);
        if (axisLength < EPS) return false;

        scalar = dot(v, axis) / axisLength;
        return true;
    }

    Static bool projectOntoAxis(const Vec3&v, const Vec3&axis, Vec3&projection) {
        double axisLengthSq = dot(axis, axis);
        if (axisLengthSq < EPS) return false;

        projection=axis* (dot(v, axis) / axisLengthSq);
        return true;
    }
};

```

В даному фрагменті коду можна побачити, що довжина, скалярний та векторний добуток, а також мішаний добуток та кут між векторами реалізовано окремими функціями.

На зображенні, який показано нижче, в наявності є окремі блоки для поточних операцій, де міститься також підказки клавіш для роботи з операціями над векторами. Для базових операцій блок підписаний як Basic operations, а для розширених - Products.



Рисунок 4. Вибір операцій над векторами у програмному проєкті.

Безпосередня побудова виконується в модулі **OperationEngine**, де аналізується поточний режим роботи, а також беруться вектори з моделі сцени і виконується

обчислення, де після цього записується результат відповідно до списків: *resultArrows*, *resultSurface*, *resultArcs* та *resultLines*.

Перед виконанням операцій, де використовується пара векторів, перевіряється, чи введені обидва вектори. Для операцій, які потребують вже три вектора, перевіряється наявність всіх трьох векторів. Це забезпечує коректність роботи та правильність з точки зору математики.

Базові операції, такі як: додавання, віднімання, множення на скаляр, формують не лише числовий результат, а й побудову графічних об'єктів для подальшого відображення:

```
case OperationMode::AddParallelogram: {
    scene_.operationTitle = "Vector addition: parallelogram rule";
    scene_.formulaLine = pairFormulaName("+") + " = coordinate-wise sum";
    scene_.lessonLine = "Translated copies keep direction and length; the diagonal
is the sum.";
    if (!requireAB()) break;
    Vec3 sum = u + v;
    appendLine(pairFormulaName("+") + " = " + vecToString(sum));
    appendLine("|" + pairFormulaName("+") + " | = " + fmt(vectorLength(sum), 4));

    if (!samePoint(V.start, origin)) {
        addResultArrow(origin, vEnd, V.color, secondName + "_0", 2.4f, true);
    }
    addResultArrow(origin, origin + sum, Color{0.96f, 0.88f, 0.18f, 1.0f},
pairFormulaName("+"), 4.0f);
    addResultArrow(origin + u, origin + u + v, V.color, secondName + "'", 2.5f,
true);
    addResultArrow(origin + v, origin + v + u, U.color, firstName + "'", 2.5f,
true);
    addSurface({ origin, uEnd, origin + u + v, vEnd }, Color{0.96f, 0.88f, 0.18f,
0.18f});
    break;
}
```

В даному фрагменті показано результат суми обчислення двох векторів, після цього до сцени додається результатний вектор та допоміжна поверхня. Таким чином, операція має поєднання між числовим значення та геометричним змістом.

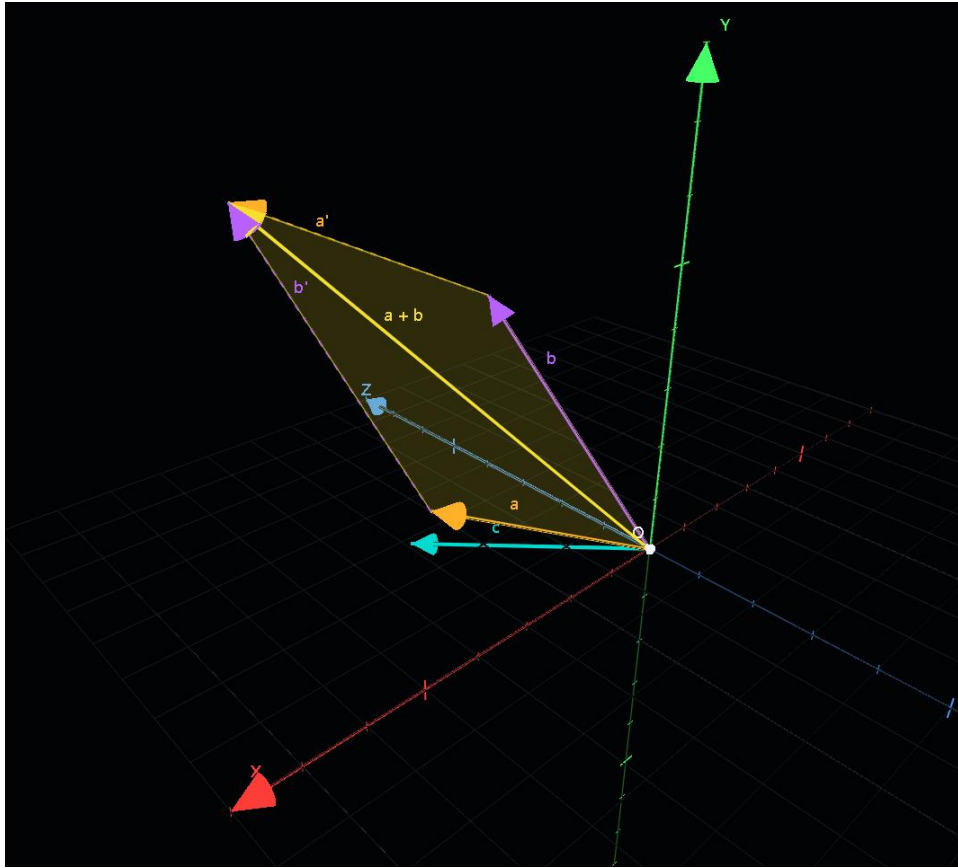


Рисунок 5. Візуальне відображення операції додавання за правилом трикутника.

Для віднімання використовується аналогічна логіка, але результат формується як різниця двох векторів, де додатково відображається протилежний напрям другого вектора.

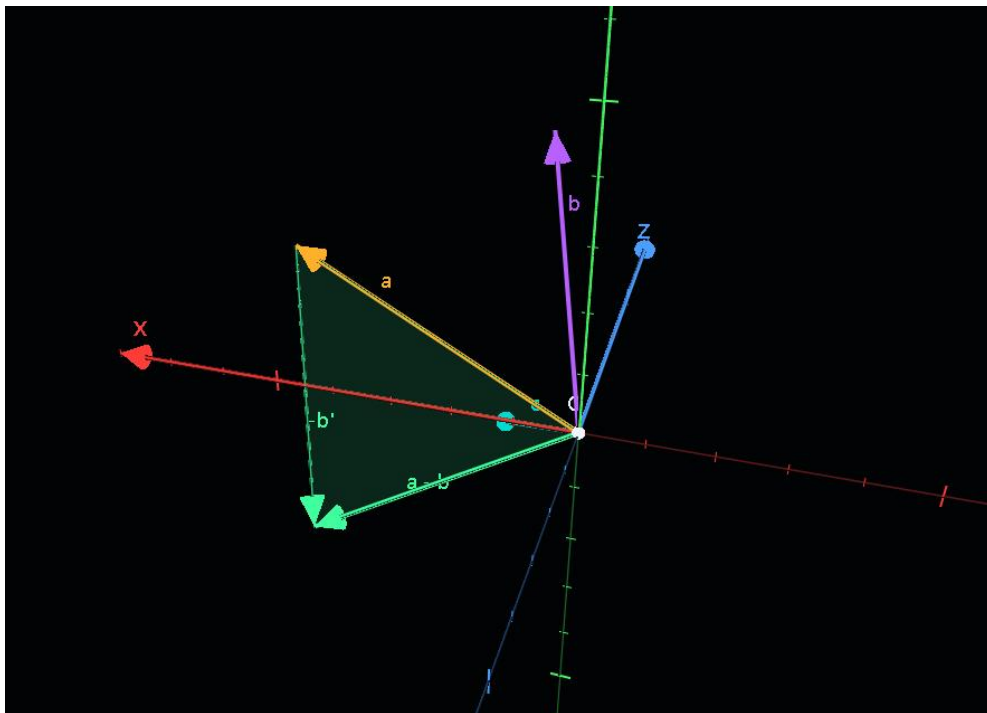


Рисунок 6. Візуальне відображення операції віднімання.

Під час множення на скаляр вибраний один вектор масштабується на задане число, яке задав користувач, а результат відображається як окремий вектор. Але якщо коефіцієнта k немає значення, тоді виставляється значення по замовчуванню.

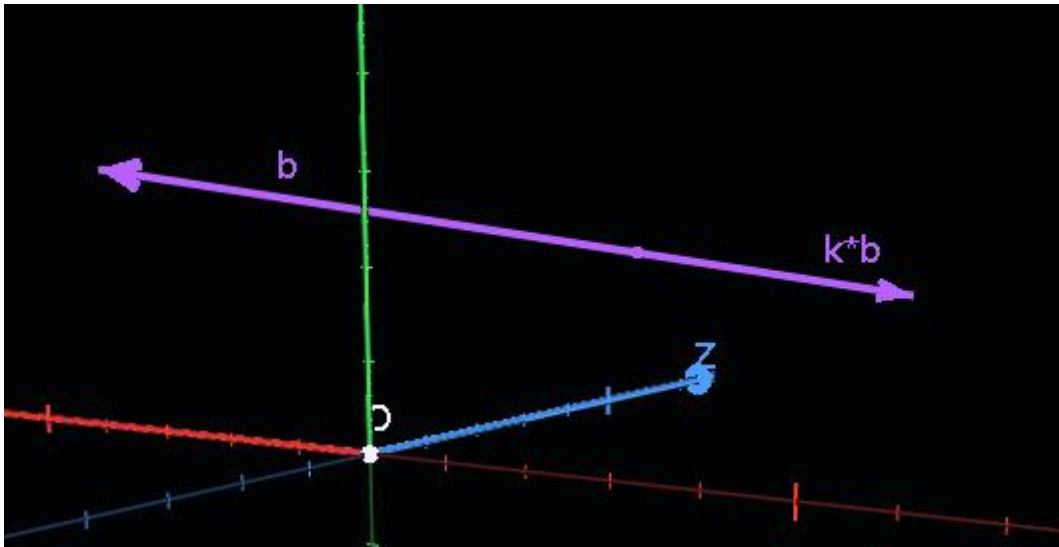


Рисунок 7. Візуалізація вектора та його вектора помноженого на скаляр.

Операції векторного та мішаного добутку, визначення проекції, мають складнішу логіку, оскільки потребують додаткових перевірок та побудови допоміжних елементів. Наприклад, під час визначення проєкції одного вектора на інший, програма визначає як саму проєкцію, так і його перпендикулярну основу:

```
case OperationMode::Projection: {
    scene_.operationTitle = "Projection of vector " + secondName + " on vector " +
    firstName;
    scene_.formulaLine = "pr_" + firstName + "(" + secondName + ") = (" +
    pairFormulaName("*") + ") / |" + firstName + "|";
    scene_.lessonLine = "The scalar projection measures the signed component of
    one vector along another.";
    if (!requireAB()) break;

    double scalar = 0.0;
    Vec3 projection;
    if (!scalarProjection(v, u, scalar) || !projectOntoAxis(v, u, projection)) {
        appendLine("Projection is undefined because vector " + firstName + " is
        zero.");
        break;
    }

    Vec3 residual = v - projection;
    appendLine("pr_" + firstName + "(" + secondName + ") = " + fmt(scalar, 4));
    appendLine("proj_" + firstName + "(" + secondName + ") = " +
    vecToString(projection));
    appendLine(secondName + " - proj_" + firstName + "(" + secondName + ") = " +
    vecToString(residual));
}
```

```

if (!samePoint(V.start, origin)) {
    addResultArrow(origin, vEnd, V.color, secondName + "_0", 2.4f, true);
}
addResultArrow(origin, origin + projection, Color{0.20f, 0.92f, 0.86f, 1.0f},
"proj_" + firstName + "(" + secondName + ")", 4.0f);
if (vectorLength(residual) > EPS) {
    addResultArrow(origin + projection, vEnd, Color{1.0f, 0.62f, 0.18f, 1.0f},
"perp", 2.8f, true);
    addSurface({ origin, origin + projection, vEnd }, Color{0.20f, 0.92f,
0.86f, 0.13f});
}
break;
}

```

Програма спочатку перевіряє, чи можна виконати проєктування. Якщо напрямлений вектор є нульовий, тоді операція не виконується, а користувач отримує повідомлення про некоректність такого випадку. Якщо ж обчислення можливе, тоді в сцені відображається вектор проєкції та перпендикулярна основа.

Зауваження: Для подальшого показу проєкції вектора, було відключено координатну вісь для покращення візуалізації.

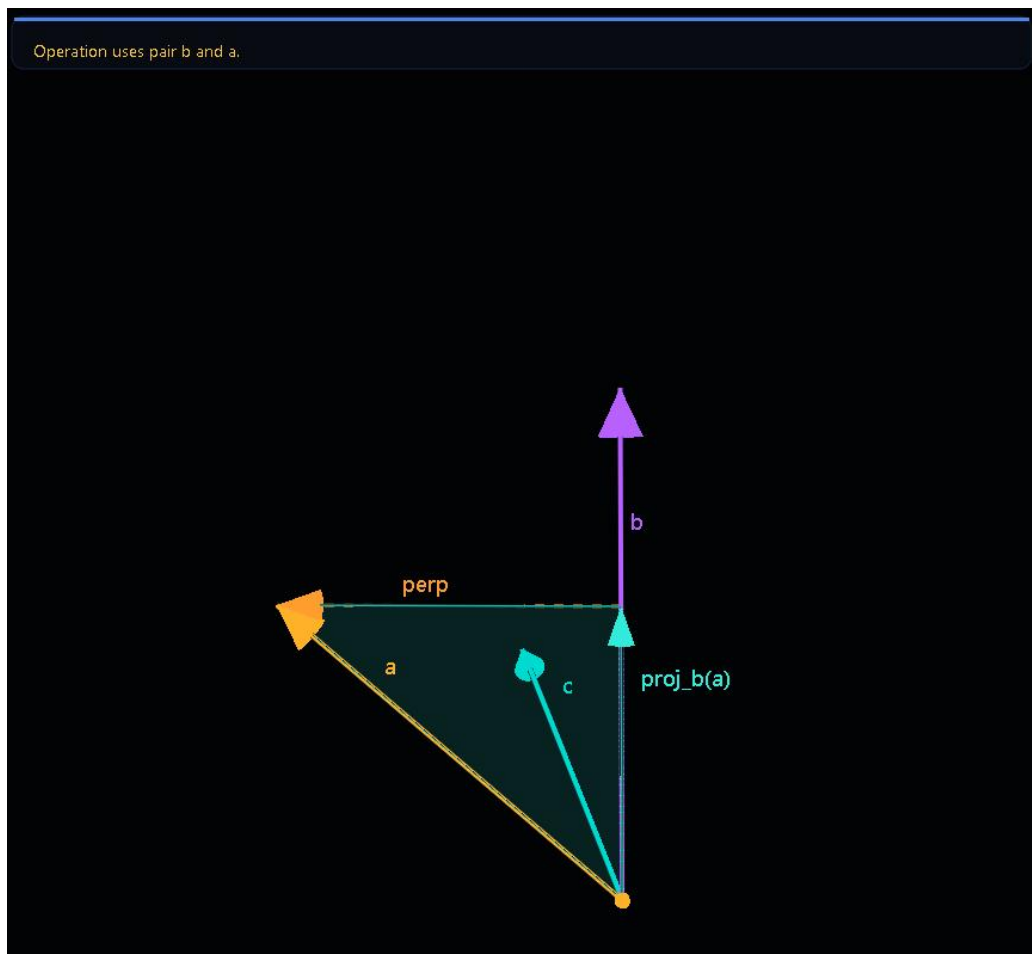


Рисунок 8. Візуалізація проєкції вектора вибраної пари та перпендикулярної основи.

Операції векторного добутку та побудови нормалі до площини мають особливе значення для комп'ютерної графіки. У програмі результат векторного добутку подається як новий вектор, перпендикулярний до пари векторів, а також додатково будується поверхня, яка показує площину утворену двома векторами:

```
case OperationMode::Cross: {
    scene_.operationTitle = "Cross product";
    scene_.formulaLine = pairFormulaName("x") + " is perpendicular to the selected
pair.";
    scene_.lessonLine = "Direction follows the right-hand rule; reversing vector
order flips the normal.";
    if (!requireAB()) break;
    Vec3 cross = crossProduct(u, v);
    appendLine(pairFormulaName("x") + " = " + vecToString(cross));
    appendLine("|" + pairFormulaName("x") + "| = " + fmt(vectorLength(cross), 4) + "
(parallelogram area)");
    appendLine("Right-hand rule: " + firstName + " x " + secondName + " sets the
normal direction.");

    if (!samePoint(V.start, origin)) {
        addResultArrow(origin, vEnd, V.color, secondName + "_0", 2.4f, true);
    }
    if (vectorLength(cross) > EPS) {
        addSurface({ origin, uEnd, origin + u + v, vEnd }, Color{1.0f, 0.44f,
0.15f, 0.18f});
        addResultArrow(origin, origin + cross, Color{1.0f, 0.52f, 0.12f, 1.0f},
pairFormulaName("x"), 4.0f);
    }
    else {
        appendLine("The cross product is zero: vectors are parallel or one is
zero.");
    }
    break;
}
```

В даному фрагменті коду, ми бачимо додаткову перевірку через *EPS*. Якщо довжина результату векторного добутку мала, або нульова, тоді програма не будує нормаль, оскільки вибрані вектор є паралельними або один із них нульовий, що дозволяє уникнути некоректної візуалізації.

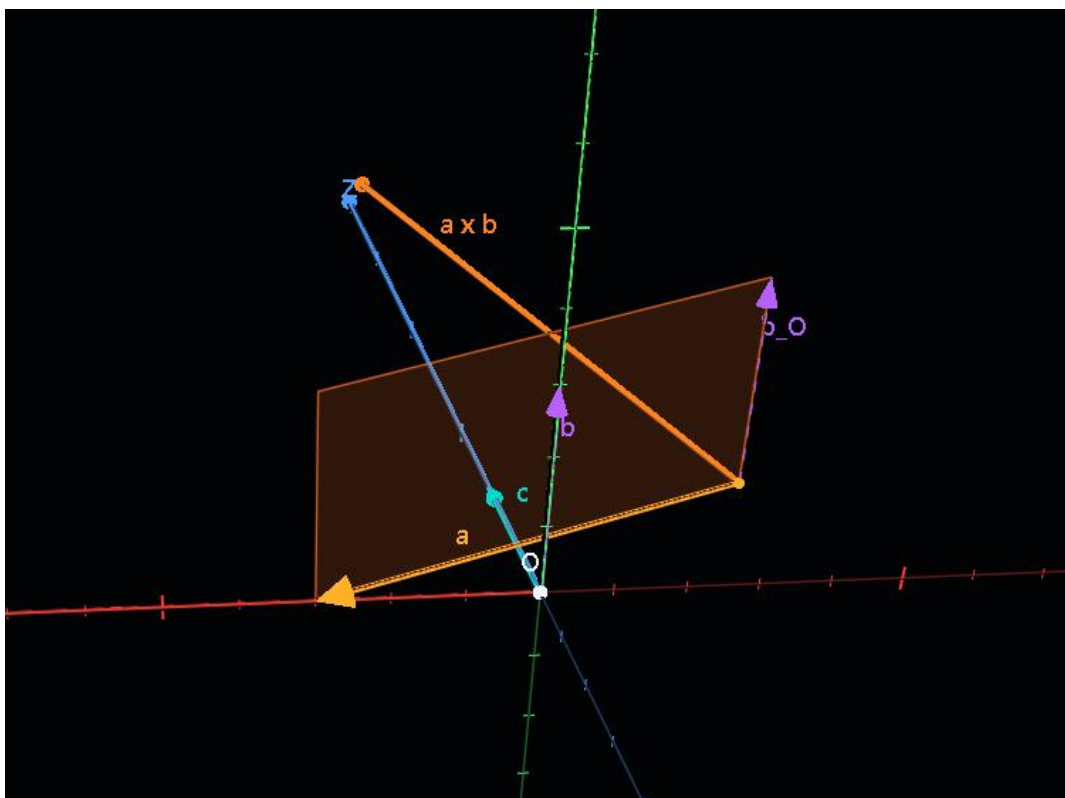


Рисунок 9. Візуалізація векторного добутку та його допоміжні елементи.

Для мішаного добутку потребується три вектори, де програма обчислює значення за формулою, виводить числовий результат та будує паралелепіпед, об'єм якого відповідає модулю мішаного добутку:

```
case OperationMode::Mixed: {
    scene_.operationTitle = "Mixed product and volume";
    scene_.formulaLine = "[a,b,c] = a*(b x c), uses all three vectors";
    scene_.lessonLine = "This operation always uses vectors a, b, and c, not only
the active pair.";
    if (!requireABC()) break;

    Vec3 bxC = crossProduct(b, c);
    double mixed = mixedProduct(a, b, c);
    appendLine("Uses vectors a, b and c.");
    appendLine("b x c = " + vecToString(bxC));
    appendLine("[a,b,c] = " + fmt(mixed, 4));
    appendLine("Volume = |[a,b,c]| = " + fmt(std::fabs(mixed), 4));
    if (!samePoint(B.start, mixedOrigin)) {
        addResultArrow(mixedOrigin, bEnd, B.color, "b_0", 2.4f, true);
    }
    if (!samePoint(C.start, mixedOrigin)) {
        addResultArrow(mixedOrigin, cEnd, C.color, "c_0", 2.4f, true);
    }
    addParallelepiped(mixedOrigin, a, b, c, Color{0.35f, 0.72f, 1.0f, 0.12f});
    addResultArrow(mixedOrigin, mixedOrigin + bxC, Color{0.90f, 0.82f, 0.20f,
1.0f}, "b x c", 3.5f);
    addResultArrow(mixedOrigin, cEnd, C.color, "c", 3.0f);
    break;
}
```

}

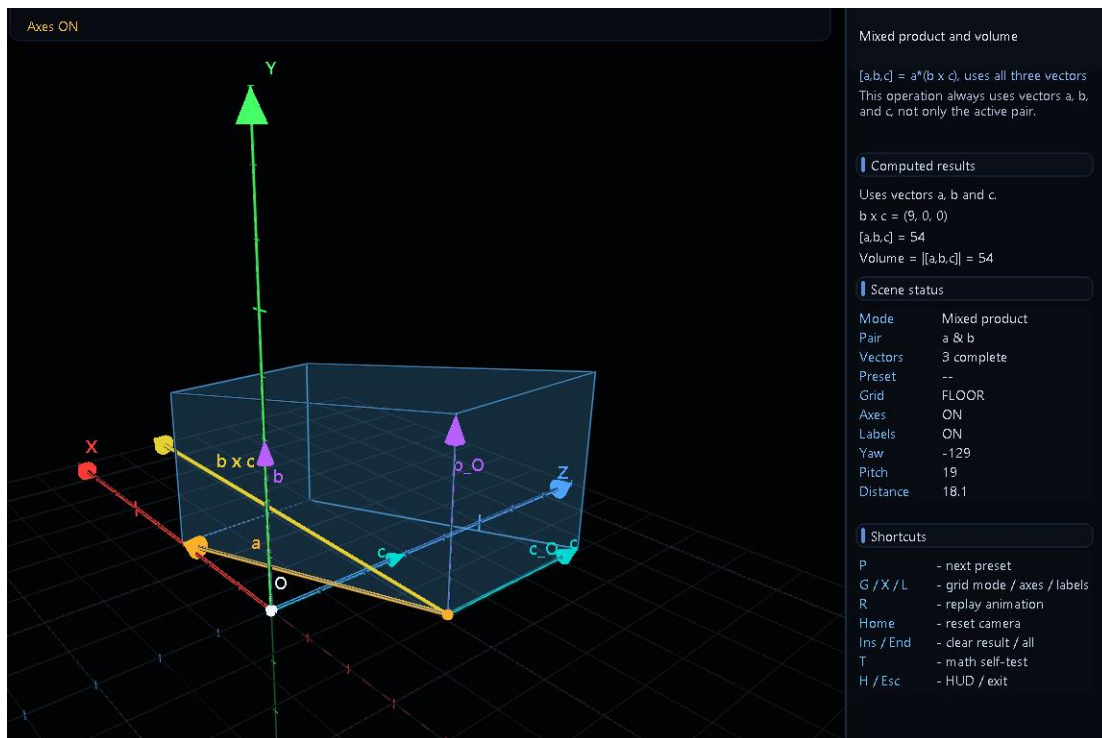


Рисунок 10. Візуалізація мішаного добутку та його результату.

У цьому випадку результат має декілька рівнів подання: текстова частина (показує значення мішаного добутку та його об'єм) та графічна (відображає паралелепіпед та вектор $\vec{b} \times \vec{c}$).

Усі результати операцій зберігаються як набір даних для сцени. Стрілки додаються через *addResultArrow*, а поверхні - через *addSurface*, дуги кутів - *resultArcs*, а текстові поняття через *resultLines*. Завдяки цьому одна й та сама логіка формує різні типи візуальних результатів в програмному застосунку.

Такий підхід спрощує оновлення сцени. Наприклад, якщо користувач змінює координати вектора або вибирає іншу операцію, тоді попередні результати очищуються після чого *OperationEngine* заново формує актуальні дані. Це забезпечує коректність між математичним результатом, пояснення у вигляді тексту та графічним відображенням на сцені.

2.5. Візуалізація сцени за допомогою графічного API OpenGL.

Формування тривимірної сцени у здійснюється за допомогою графічного API OpenGL з використанням бібліотеки GLUT. Даний графічний API використовується для відображення координатної системи, векторів, допоміжних графічних елементів, які формуються під час роботи певної математичної операції над векторами. На відміну від попередніх підрозділів, в даному підрозділі розглядає саме процес виведення результатів та графічних елементів на екран. Графічна частина не визначає

математичний зміст операції, а лише відповідає за їх задання в тривимірному просторі у зрозумілій формі для користувача.

Процес побудови сцени можна розглядати наступним чином. Нижче наведена таблиця для кращої зрозумілості:

Етап побудови	Призначення
Ініціалізація вікна	Створення вікна та налаштування режиму відображення
Налаштування OpenGL	Увімкнення глибини, прозорості, освітлення сцени та загладжування ліній
Налаштування камери	Встановлення перспективної проєкції та положення напрямку і координат камери
Очищення кадру	Очищення буферних даних кольору та глибини перед наступним новим кадром
Побудови сцени	Малювання координатної осі, осей, а також векторів та інших графічних елементів
Виведення інтерфейсу	Малювання інформаційної панелі з виведенням результатів
Заміна даних буферів	Виведення готового кадру на екран

Таблиця 4. Основні етапи побудови тривимірної сцени.

Ініціалізація графічного середовища виконується в окремому класі *GlutApp*. Саме цей компонент дозволяє створити вікно, задає режим буферизації, підключає обробники подій клавіатури та миші, оновлює кадри та зміни розміру екрана для оптимізації:

```
int GlutApp::run(int argc, char** argv) {
    activeApp_ = this;
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA | GLUT_DEPTH);
    glutInitWindowSize(width_, height_);
    glutCreateWindow("3D Vector Visualizer");

    initOpenGL();

    sceneController_.rebuildOperation();
    sceneController_.resetAnimation();

    glutDisplayFunc(displayCallback);
    glutReshapeFunc(reshapeCallback);
    glutKeyboardFunc(keyboardCallback);
    glutSpecialFunc(specialKeyboardCallback);
    glutMouseFunc(mouseButtonCallback);
}
```

```

glutMotionFunc(mouseMotionCallback);
glutTimerFunc(16, timerCallback, 0);

printConsoleHelp();
glutMainLoop();
return 0;
}

```

В даному фрагменті коду використовується **GLUT_DOUBLE**, який дозволяє вмикати буферизацію. Дана буферизація є подвійною, оскільки кадр спочатку формується у прихованому буфері, а пізніше виводиться на екран, що дозволяє зменшити “шум” під час оновлення сцени. Також тут використовується **GLUT_DEPTH**, який додає буфер глибини відображення об’єктів у просторі.

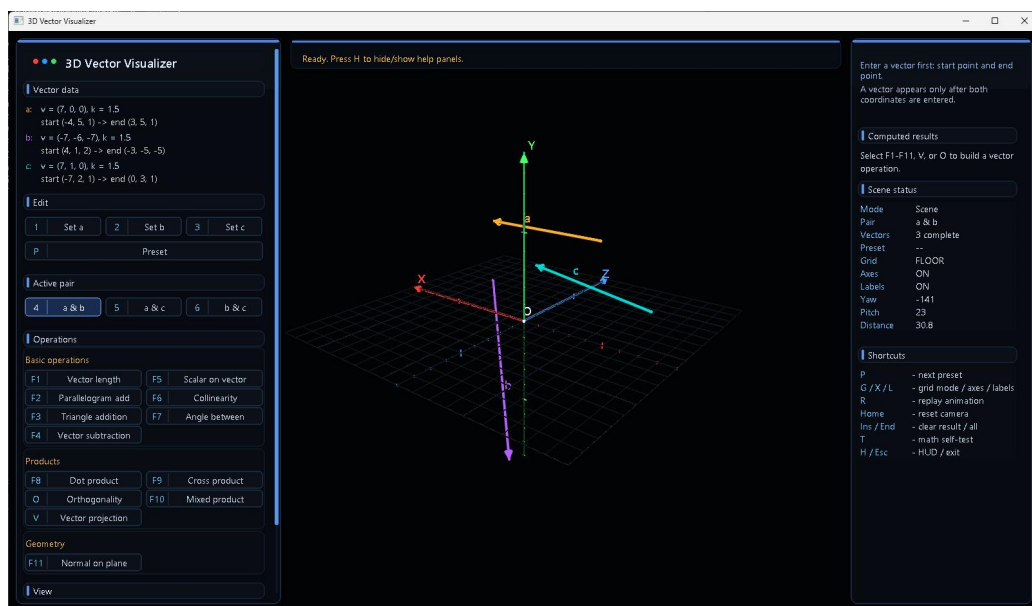


Рисунок 11. Загальний вигляд вікна програмного застосунку.

Після створеного вікна, використовується налаштування базових параметрів **OpenGL**. У програмі в наявності присутня перевірка глибини, прозорості, освітлення та матеріальні властивості кольору. Це дозволяє користувачу побачити коректність відображення об’єктів, які розташовані на різній відстані від камери, а також намалювати напівпрозорі площини:

```

void GlutApp::initOpenGL() {
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glEnable(GL_COLOR_MATERIAL);
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);

    GLfloat lightPos[] = { 6.0f, 12.0f, 8.0f, 1.0f };
    GLfloat ambient[] = { 0.18f, 0.20f, 0.24f, 1.0f };
    GLfloat diffuse[] = { 0.85f, 0.88f, 0.92f, 1.0f };
    glLightfv(GL_LIGHT0, GL_POSITION, lightPos);
    glLightfv(GL_LIGHT0, GL_AMBIENT, ambient);
}

```

```

glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse);

textRenderer_.initFonts();
hudRenderer_.initFonts();
}

```

В даному фрагменті коду можна побачити цікавий об'єкт під назвою **GL_DEPTH_TEST**. Даний об'єкт забезпечує правильний порядок перекриття об'єктів, якщо один вектор або площина розташовані позаду іншого елементи сцени. Також можна побачити **GL_BLEND**, який використовується для прозорих поверхонь, наприклад площину, які демонструють паралелограм, проєкцію, тощо.

Також невід'ємною частиною являється камера. У програмного проєкті камера використовується як перспективна проєкція, яка створює звичне для тривимірної графіки. Для цього задається кут огляду, співвідношення сторін вінка та ближня й дальня площини відсікання:

```

void GlutApp::setupCamera() {
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    double aspect = height_ == 0 ? 1.0 : static_cast<double>(width_) / static_cast<double>(height_);
    gluPerspective(55.0, aspect, 0.05, 500.0);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    Vec3 eye = camera_.eye();
    gluLookAt(eye.x, eye.y, eye.z, camera_.target.x,
    camera_.target.y, camera_.target.z, 0, 1, 0);
}

```

Ми можемо звернути увагу на функцію **glutPerspective()**, в якій головна роль задання перспективної проєкції, а функція **glutLookAt()** встановлює положення камери та точку, на яку вона спрямована. Також використовується програмний об'єкт **eye**, який визначає позицію спостерігача, а **camera.target** - центр огляду. Завдяки цим елементам, користувач може споглядати одну й ту саму геометричну побудову з різних ракурсів.

Також для зручної роботи з векторами, для кожної побудови кадру використовується метод **display()**. На початку кадру задається область виведення, очищуються буфери кольору та глибини, і після чого викликається налаштування камери. Далі окремо зображається тривимірна геометрична сцена та інтерфейсу панелі користувача (HUD):

```

void GlutApp::display() {
    glViewport(0, 0, width_, height_);
    glClearColor(0.008f, 0.012f, 0.020f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glEnable(GL_DEPTH_TEST);
    glEnable(GL_BLEND);
}

```

```

glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glEnable(GL_LINE_SMOOTH);
glHint(GL_LINE_SMOOTH_HINT, GL_NICEST);

setupCamera();
sceneRenderer_.draw();
hudRenderer_.draw(width_, height_);

glutSwapBuffers();
}

```

Цей фрагмент коду показує загальну послідовність побудови кадру, де спочатку відображаються геометричні об'єкти, після цього - елементи інтерфейсу. Наприкінці викликається *glutSwapBuffers()*, який переносить кадр із заднього буфера на екран.

Уже за малювання геометрії відповідає *SceneRenderer*. Саме він бере на себе відповідальність вже за підготовлені дані зі сцени: користувацькі та результуючі вектори, поверхні та дуги і інші допоміжні геометричні елементи:

```

void SceneRenderer::draw() const {
double limit = sceneLimit();

    glPushAttrib(GL_ENABLE_BIT | GL_CURRENT_BIT | GL_LINE_BIT);
    glEnable(GL_DEPTH_TEST);

    if (scene_.ui.gridMode != GridMode::Off) {
        drawGridPlane(limit, 1.0, 0);
    }
    if (scene_.ui.gridMode == GridMode::Full) {
        drawGridPlane(limit, 2.0, 1);
        drawGridPlane(limit, 2.0, 2);
    }

    drawAxes(limit);

    for (const auto& surface : scene_.resultSurfaces) {
        drawSurface(surface);
    }

    double t = smoothStep(scene_.ui.animationT);

    std::vector<Arrow3D> labelArrows;
    std::array<int, 3> vectorLabelLanes = { 0, 0, 0 };

    for (const auto& slot : scene_.vectors) {
    if (!slot.complete()) continue;
        labelArrows.push_back(Arrow3D{ slot.start, slot.end, slot.color, slot.name,
4.0f, false });
    }

    if (scene_.ui.showGhosts) {
        labelArrows.insert(labelArrows.end(), scene_.resultArrows.begin(),
scene_.resultArrows.end());
    }
}

```

```

    }

    assignLabelLanes(labelArrows);

    size_t laneIndex = 0;
    for (size_t i = 0; i < scene_.vectors.size(); ++i) {
        if (!scene_.vectors[i].complete()) continue;
        vectorLabelLanes[i] = labelArrows[laneIndex++].labelLane;
        drawVectorSlot(scene_.vectors[i], 1.0, vectorLabelLanes[i]);
    }

    if (scene_.ui.showGhosts) {
        for (size_t i = 0; i < scene_.resultArrows.size(); ++i) {
            Arrow3D arrow = scene_.resultArrows[i];
            if (laneIndex + i < labelArrows.size()) {
                arrow.labelLane = labelArrows[laneIndex + i].labelLane;
            }
            drawArrow(arrow, t);
        }
    }

    for (const auto& arc : scene_.resultArcs) {
        drawArc(arc);
    }

    glPopAttrib();
}

```

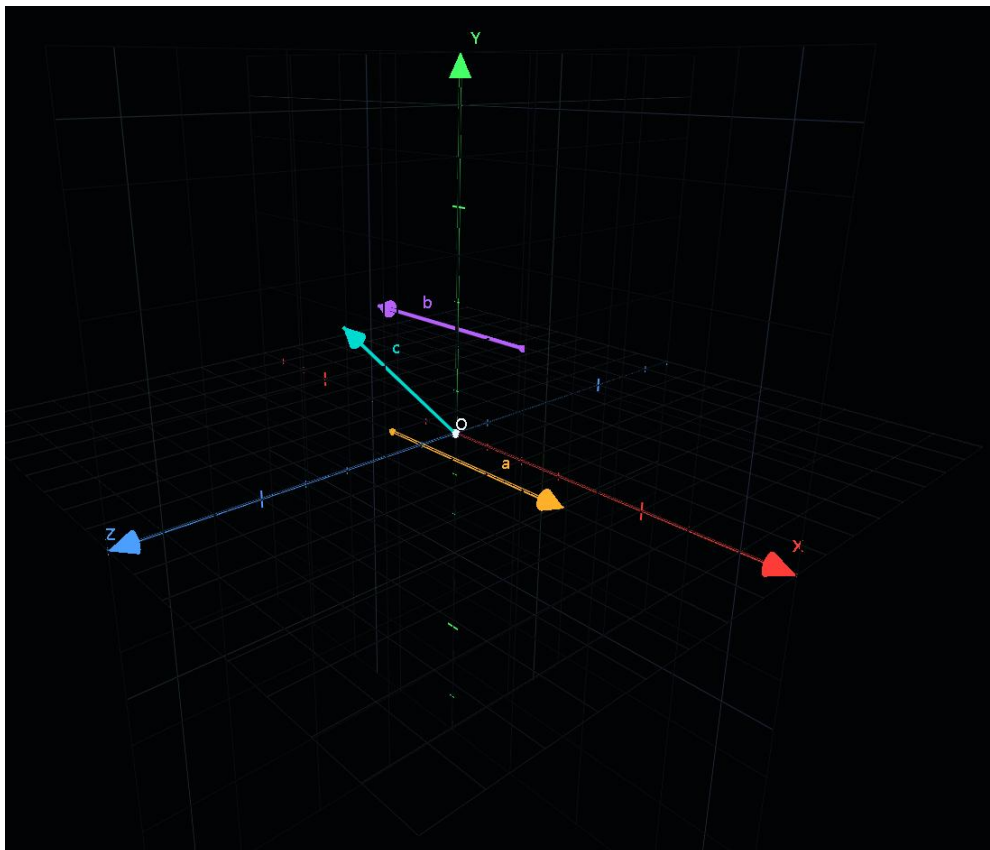


Рисунок 12. Координатна сітка, осі та вектори.

На зображенні можна розглянути координатну сітку, осі та три вектора, призначені для подальших операцій з ними. Але окремо увагу розглянемо саме вектор. Вектор в геометричному сенсі відображають як напрямлену лінію з наконечником, в результаті отримуємо вектор у вигляді стріли. Стріла складається з лінійного відрізка та конуса, який візуально показує напрям, що для правильної орієнтації є коректним. Також для орієнтації конуса використовується напрямний одиничний вектор:

```
void SceneRenderer::drawArrow(const Arrow3D&arrow, double progress) const {
    Vec3 delta = arrow.end - arrow.start;
    Vec3 partialEnd = arrow.start + delta * progress;
    Vec3 dir = partialEnd - arrow.start;
    double length = dir.length();
    if (length < EPS) {
        glPushAttrib(GL_ENABLE_BIT | GL_CURRENT_BIT);
        glDisable(GL_LIGHTING);
        setColor(arrow.color);
        glPushMatrix();
        glTranslated(arrow.start.x, arrow.start.y, arrow.start.z);
        glutSolidSphere(0.13, 18, 12);
        glPopMatrix();

        if (!arrow.label.empty() && drawText3D_) {
            int signature = 0;
            for (unsignedchar ch : arrow.label) signature += ch;
            double lane = static_cast<double>((signature % 5) - 2) * 0.45
                + static_cast<double>(arrow.labelLane) * 1.20;
            double lift = static_cast<double>(((signature / 5) % 3) - 1);
            Vec3 right = camera_.right();
            Vec3 up = right.cross(camera_.forward()).normalized();
            Vec3 labelPos = arrow.start
                + right * (0.24 * lane)
                + up * (0.30 + 0.10 * lift + 0.08 * std::abs(arrow.labelLane));
            drawText3D_(labelPos, arrow.label + " = 0", arrow.color);
        }

        glPopAttrib();
    }
    return;
}

Vec3 unit = dir / length;

    glPushAttrib(GL_LINE_BIT | GL_ENABLE_BIT | GL_CURRENT_BIT);
    glDisable(GL_LIGHTING);
    if (arrow.dashed) {
        glEnable(GL_LINE_STIPPLE);
        glLineStipple(1, 0x00FF);
    }

    glLineWidth(arrow.width);
    setColor(arrow.color);
    glBegin(GL_LINES);
```

```

    glVertex3d(arrow.start.x, arrow.start.y, arrow.start.z);
    glVertex3d(partialEnd.x, partialEnd.y, partialEnd.z);
    glEnd();

    double coneHeight = std::clamp(length * 0.13, 0.18, 0.48);
    double coneRadius = std::clamp(length * 0.045, 0.07, 0.22);
    Vec3 coneBase = partialEnd - unit * coneHeight;

    glPushMatrix();
    glTranslated(coneBase.x, coneBase.y, coneBase.z);

    Vec3 base(0, 0, 1);
    Vec3 axis = base.cross(unit);
    double axisLen = axis.length();
    if (axisLen > EPS) {
        axis = axis / axisLen;
        double angle = std::acos(std::clamp(base.dot(unit), -1.0, 1.0)) * 180.0 / PI;
        glRotated(angle, axis.x, axis.y, axis.z);
    }
    elseif (unit.z < 0) {
        glRotated(180.0, 1.0, 0.0, 0.0);
    }

    glutSolidCone(coneRadius, coneHeight, 24, 12);
    glPopMatrix();

    if (!arrow.label.empty() && drawText3D_) {
        int signature = 0;
        for (unsignedchar ch : arrow.label) signature += ch;
        double lane = static_cast<double>((signature % 5) - 2) * 0.45
            + static_cast<double>(arrow.labelLane) * 1.20;
        double lift = static_cast<double>(((signature / 5) % 3) - 1);
        double labelAt = (arrow.label == "X" || arrow.label == "Y" || arrow.label == "Z") ?
            0.98 : 0.68;
        Vec3 right = camera_.right();
        Vec3 up = right.cross(camera_.forward()).normalized();
        Vec3 labelPos = arrow.start + (partialEnd - arrow.start) * labelAt
            + right * (0.22 * lane)
            + up * (0.24 + 0.09 * lift + 0.06 * std::abs(arrow.labelLane));
        drawText3D_(labelPos, arrow.label, arrow.color);
    }

    glPopAttrib();
}

```

Можна запримітити, що графічне подання вектора також спирається на векторній операції. Для орієнтації конуса використовується векторний добуток між базовим напрямком вектора (0,0,1) та напрямком стрілки. Кут повороту визначається через скалярний добуток.

Особливу увагу також потрібно звернути на відображення площини і допоміжних поверхонь, де використовується напівпрозоре заливання. Це потрібно для того, щоб поверхня не закривала повністю інші елементи сцени:

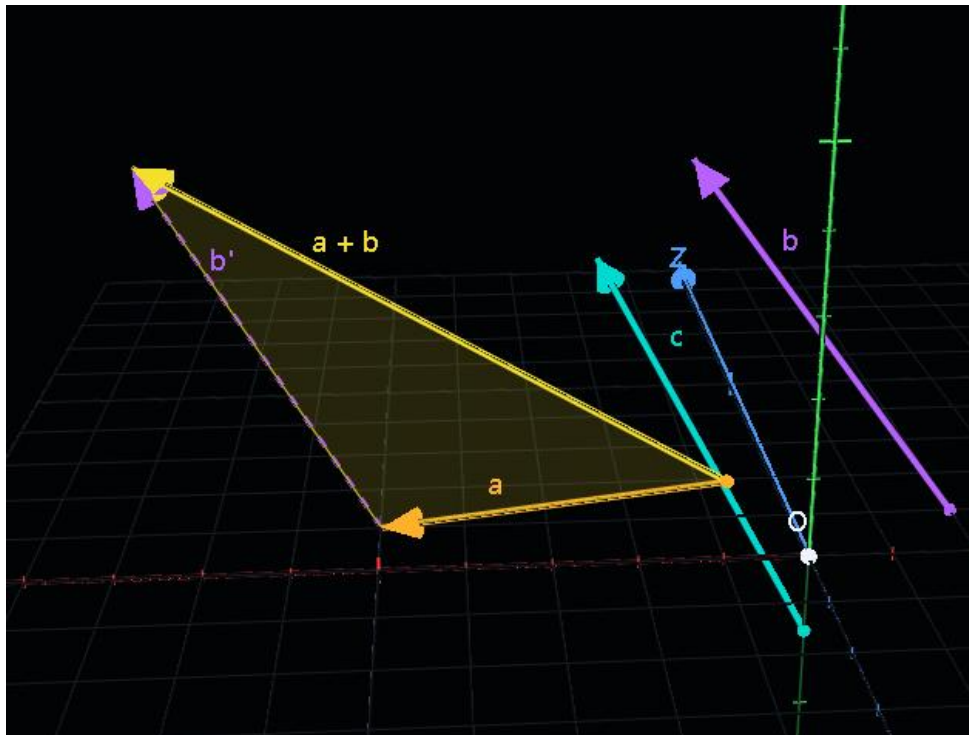


Рисунок 13. Приклад використання прозорих поверхонь.

У результаті користувач бачить не лише початкові вектори, а й саму область між ними, що робить режим обчислення кута більш наочним. Таким чином, побудова тривимірної сцени у програмному забезпеченні реалізована як послідовність окремих графічних елементів, дозволяє відокремити математичні обчислення від рендерингу та забезпечує зрозуміле відображення операцій над векторами у тривимірному просторі.

2.6. Візуалізація операцій з векторами

Після виконання математичних обчислень важливо подати результат у такій формі, щоби користувач міг швидко зрозуміти його геометричний зміст. Для цього у програмі використовується система візуальних позначень. Візуалізація побудова не як просте малювання певних ліній, а як набір графічних примітивів, в якого своє призначення:

Елемент	Спосіб відображення	Призначення
Початковий вектор	Кольорова суцільна стрілка	Вектор, введений користувачем
Результуючий вектор	Товстіша стрілка іншого вектора	Результат вибраної операції
Перенесений вектор	Пунктирна стрілка	Допоміжна копія вектора

Площина або паралелограм	Напівпрозора поверхня	Пояснення просторової побудови
Нормаль	Окрема стрілка з підписом N	Перпендикуляр до площини
Кут	Дуга між двома векторами	Наочне подання кута
Проекція	Стрілка вздовж напрямленого вектора	Паралельна складова
Перпендикулярна складова	Пунктирна допоміжна стрілка	Частина вектора після віднімання проекції

Таблиця 5. Основні етапи побудови тривимірної сцени.

Для формування результатів використовується спеціальні допоміжні методи, які не малюють об'єкт зразу, а додають їх до списків моделі сцени. Після чого рендерер зчитує ці списки та відображає відповідні елементи:

```
void OperationEngine::addResultArrow(const Vec3&start, const Vec3&end, const
Color&color, const std::string&label, float width, bool dashed) {
    scene_.resultArrows.push_back(Arrow3D{ start, end, color, label, width,
dashed });
}

void OperationEngine::addSurface(const std::vector<Vec3>&vertices, const
Color&color, bool outline) {
    scene_.resultSurfaces.push_back(Surface3D{ vertices, color, outline });
}
```

Такий підхід дозволяє відокремити логіку операції від її графічного задання на сцені. Для звичайних векторів є стрілка. Колір допомагає розрізняти між собою вектори, а підпис біля стрілки швидко зрозуміти, який вектор на сцені зображено.

Під час додавання або віднімання програма використовує додаткові стрілки, які показують перенесені копії вектора та дозволяє пояснити за геометричним значення правила трикутника чи паралелограма.

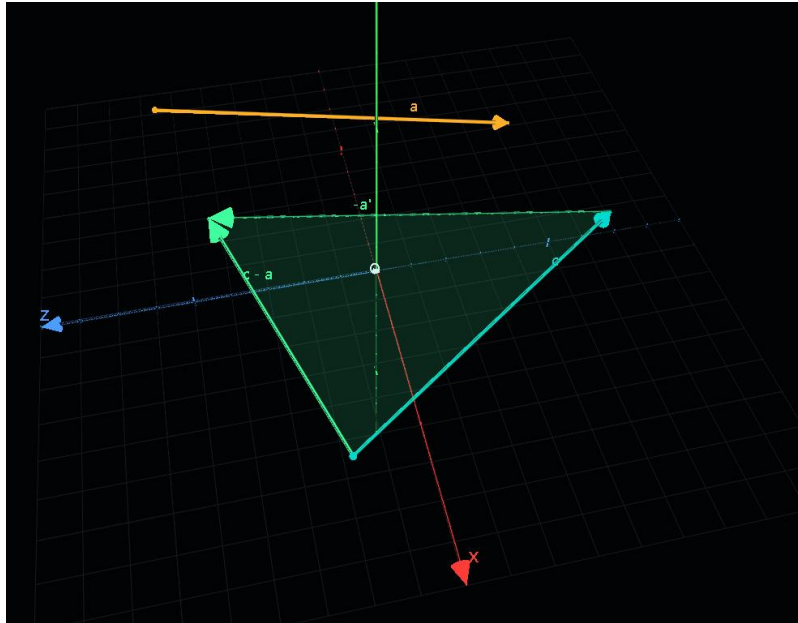


Рисунок 13. Відображення допоміжних стрілок та площин під час операцій.

Площини у програмі використовуються як допоміжні об'єкти. Вони не є самостійною метою візуалізації, але допомагають пояснити просторові зв'язки між певними векторами. Наприклад, у векторного добутку прозора площина показує область, у якій лежать два початкові вектори, а нормаль відображає як вектор, який є перпендикуляром до цієї площини. Або наприклад, для нормалі використовується окрема стрілка з підписом N , що надає змогу візуально побачити відокремлення її від початкових векторів та підкреслити, що саме вона є результатом побудови.

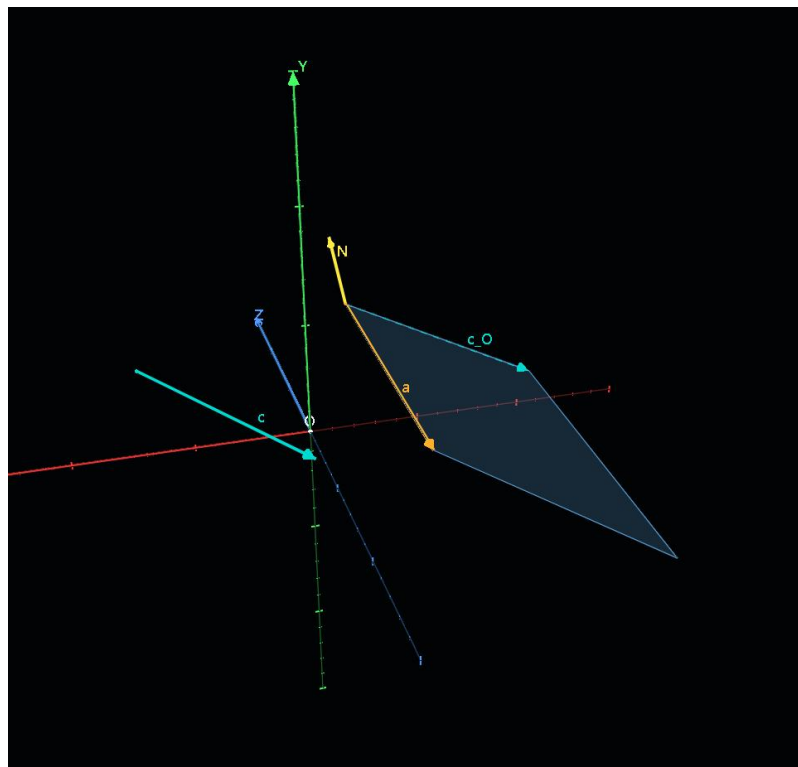


Рисунок 14. Побудова нормалі до площини.

Окремий спосіб відображення використовується також для кутів між двома векторами, який показується не лише числовим значенням, а й дугою у просторі. Це дозволяє правильно зрозуміти між якими саме напрямками використовується вимірювання.

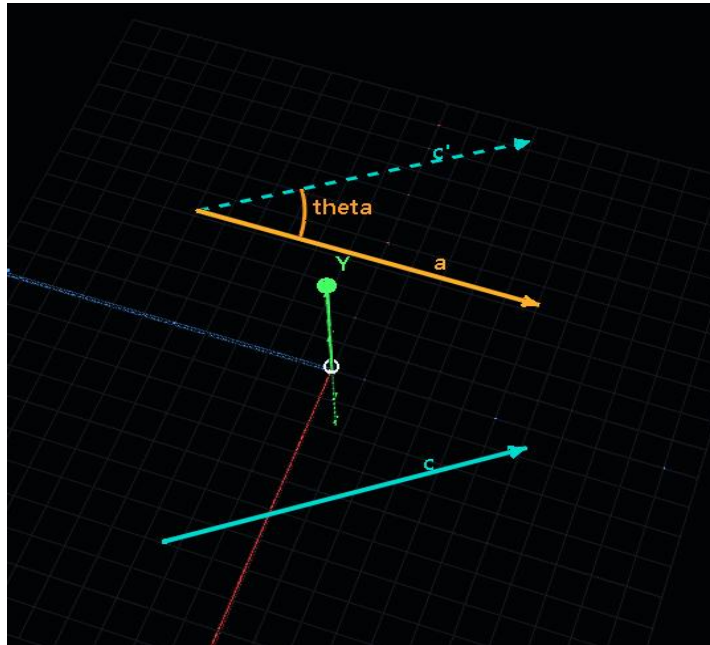


Рисунок 15. Відображення кута між векторами.

Наступним розглянемо проєкцію вектора. Вона подається через дві складові: власне проєкція та залишок. Проєкція малюється вздовж напрямного вектора, а залишкова складова відображається як допоміжна стрілка, що дозволяє візуально добре побачити, що початковий вектор можна розкласти на паралельну та перпендикулярну частини.

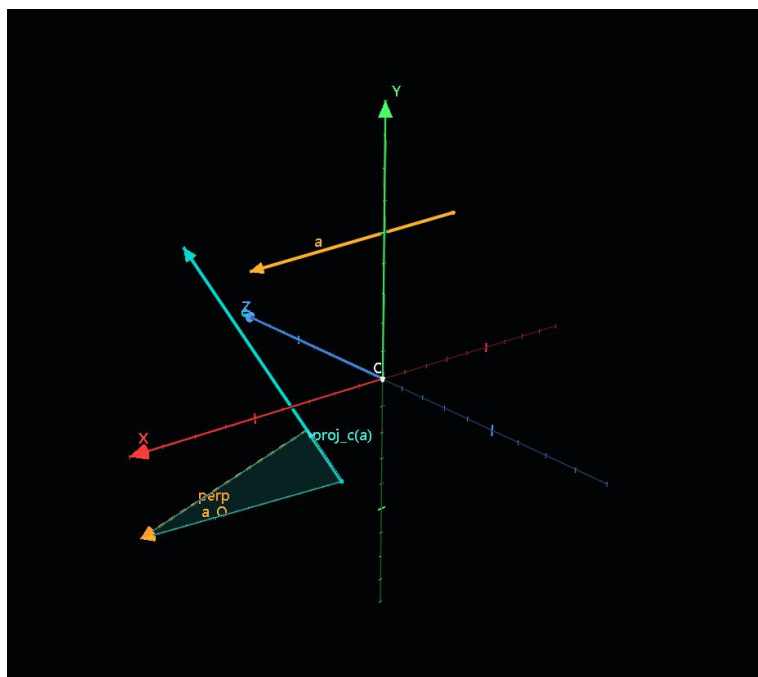


Рисунок 16. Відображення проєкції вектора.

Також розглянемо мішаний добуток, для якого використовується об'ємна візуалізація, за допомогою трьох векторів, які утворюються ребра паралелепіпеда, а прозорі грані показують фігуру, об'єм якої пов'язаний зі значення мішаного добутку.

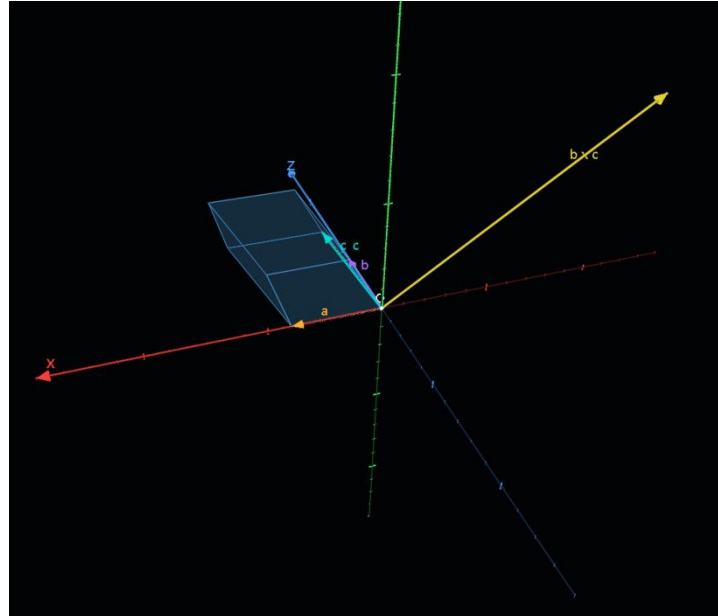


Рисунок 17. Відображення фігури утвореною мішаним добутком.

Отже, візуалізація результатів у програмі побудова як система умовних графічних позначень, де кожна операція має свій спосіб подання. Це робить програму не лише обчислювальним інструментом, а також засобом для наочного розуміння векторної алгебри в тривимірному просторі

2.7. Реалізація користувацького інтерфейсу та керування камерою.

Користувацький інтерфейс у програмному застосунку виконує не лише допоміжну роль, а також є головною частиною навчальної логіки застосунку. Його завдання полягає в тому, щоби користувач міг швидко задати вектори, вибрати потрібну операцію, побачити очікувальний результат і в разі потреби змінити ракурс перегляду сцени. А також спробувати змінити вектори для декількох прикладів для кращої зрозумілості тієї чи іншої операції.

В даному програмному забезпеченні взаємодія для введення даних та інших махінацій реалізовано обробку клавіатури та миші, що дозволяє зручно перемикатися між режимами й одразу побачити зміни в сцені. Введення векторів здійснюється через вибір одно із трьох доступних векторів ($\vec{a}$, $\vec{b}$ або $\vec{c}$). Для цього використовуються клавіші 1, 2 та 3, після чого користувач вводить спочатку початкові координати вектора, а пізніше кінцеві. Далі для вибору активної пари користувач використовує клавіші 4, 5 або 6. Детальніше можна глянути на рисунку нижче:



Рисунок 18. Інформаційна панель з підказками для користувача (ліва частина).

Інтерфейс застосунку містить інформаційну панель, на якій відображається поточний режим, координати введених векторів, активна пара, вибір операцій, тощо. На даному *рисунку 18*, лише показано половина користувацького інтерфейсу. Інша половина дозволяє побачити стан сцени, коротке пояснення, формули та результат обчислення.

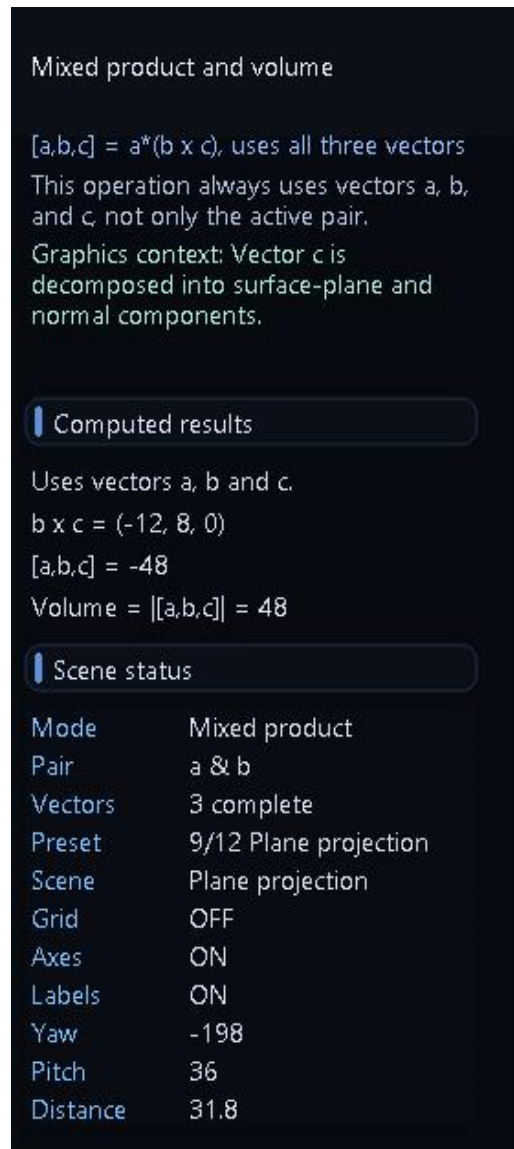


Рисунок 19. Інформаційна панель з підказками для користувача (права частина).

Окремою частиною інтерфейсу є повідомлення про стан введення. Він служить якщо користувач ввів некоректно координати, або показує стан сцени, або якщо для операцій не вистачає потрібних операцій, тощо.

Для швидкого ознайомлення з можливостями програми передбачено готові демонстраційні приклади, які можна перемкнути за допомогою клавіші **P** (англійська розкладка) і дозволяє одразу побачити різні вектори та різні операції.

Керування камерою реалізовано окремо від математичної частини програми, вона працює за принципом орбітального перегляду: камера розташована на певній відстані від центра сцени та може обертатися навколо нього.



Рисунок 20. Управління камерою, а також режимом сітки.

Також для камери використовується альтернативні, якщо в користувача, через якісь причини, немає в наявності миші або комп'ютерна миша не робоча.

Положення камери визначаються через кути повороту, відстані до центра сцени та цільову точку огляду. Для цього використовується клас **CameraController**:

```
Vec3CameraController::eye() const {
    double yawRad = yaw * PI / 180.0;
    double pitchRad = std::clamp(pitch, -86.0, 86.0) * PI / 180.0;
    double cp = std::cos(pitchRad);
    return Vec3(
        target.x + distance * cp * std::sin(yawRad),
        target.y + distance * std::sin(pitchRad),
        target.z + distance * cp * std::cos(yawRad)
    );
}
```

У цьому фрагменті кода положення обчислюється за допомогою двох кутів: *yaw* та *pitch*. Параметр *yaw* відповідає за горизонтальний поворот камери, а *pitch* - вертикальний нахил камери, також обмеження даного кута запобігає надмірному перевертанню камери. За дистанцію камери від центра сцени відповідає *distance*.

Ще одним важливим є керування допоміжних елементів сцени. Розглянемо їх детальніше.

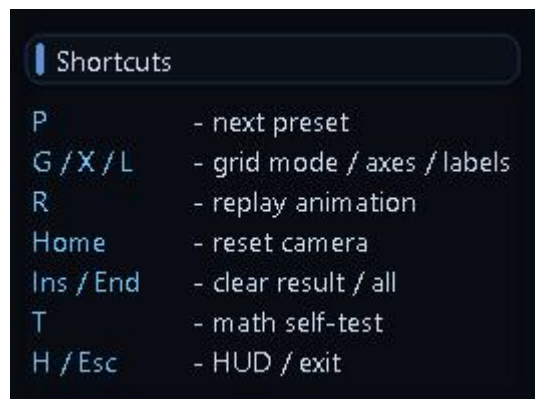


Рисунок 21. Управління допоміжними елементами інтерфейсом

Клавiша **G** перемикає режим координатної сітки (рисунок 20). Клавiша **X** відповідає за вимикання/вмикання координатної осі, **L** - за підписи, **H** - за HUD (користувацький інтерфейс). Також в наявності є повернення координат камери до початкової позиції, перевірка правильності математичних операцій, повтор анімації векторних операцій, очищення математичних операцій або самих векторів і також припинення дії програми.

Отже, користувацький інтерфейс програмного проєкту орієнтований на швидку взаємодію з тривимірною сценою, де клавіатура обробляє введення векторів, вибору операцій та перемикавання режимів, а також альтернативні дії камери та взаємодії з HUD, тоді коли миша - стандартне управління камерою. В той час інформаційна панель дозволяє побачити стан, результати, підказки зрозумілі для користувача.

2.8. Перевірка коректності роботи та демонстраційні приклади

Перевірка коректності роботи програми є важливим етапом. Дана перевірка працює у двох напрямках: перевірка математичних обчислень при виконанні операцій з векторами та перевірка правильності візуального відображення результатів у тривимірному просторі сцени. Це дозволить переконатися, що програма подає правильне графічне представлення та коректно виконує відповідні математичні обчислення.

Для перевірки математичного ядра у проєкті передбачено окремий модуль самотестування, якого роль це набір контрольних обчислень для основних операцій векторної алгебри. Далі розберемо кожний випадок з кожною операцією окремо.

Перевірка обчислення довжини вектора.

Для перевірки довжини використовується вектор, скористуємося таблицею:

Вектор	Початкова точка	Кінцева точка	Отриманий вектор
$\vec{a}$	(2; -1; 4)	(7; 3; -2)	(5; 4; -6)

Таблиця 6. Вхідні дані для перевірки довжини вектора.

Довжина вектора дорівнює:

$$|\vec{a}| = \sqrt{a_x^2 + a_y^2 + a_z^2} = \sqrt{5^2 + 4^2 + (-6)^2} = \sqrt{77} \approx 8.77$$

В результаті програма обчислила такий результат:

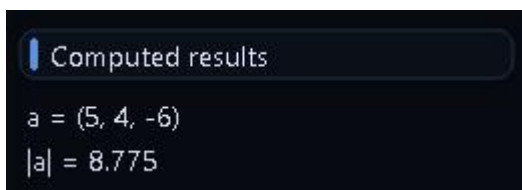


Рисунок 22. Числовий результат довжини вектора.

Перевірка обчислення суми вектора.

Для перевірки додавання перевірялося в двох режимах з однаковими координатами: правило трикутника та правило паралелограма. Скористуємося таблицею:

Вектор	Початкова точка	Кінцева точка	Отриманий вектор
$\vec{a}$	(1; -2; 3)	(5; 4; -1)	(4; 6; -4)
$\vec{b}$	(-3; 2; 1)	(2; -1; -7)	(5; -3; -6)

Таблиця 7. Вхідні дані для перевірки суми векторів.

Сума векторів дорівнює:

$$\vec{a} + \vec{b} = (a_x + b_x; a_y + b_y; a_z + b_z) = (4 + 5; 6 + (-1); (-4) + (-6)) = (9; 5; -10)$$

В результаті програма обчислила такий результат:

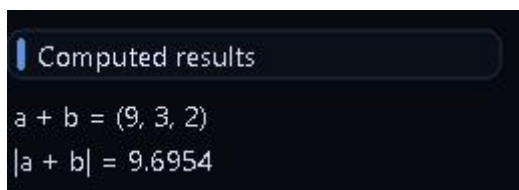


Рисунок 23. Числовий результат суми векторів та довжини суми.

З точки зору алгебри результат правильний, лишилося показати як виглядає дана операція геометрично правилом паралелограма та правилом трикутника.

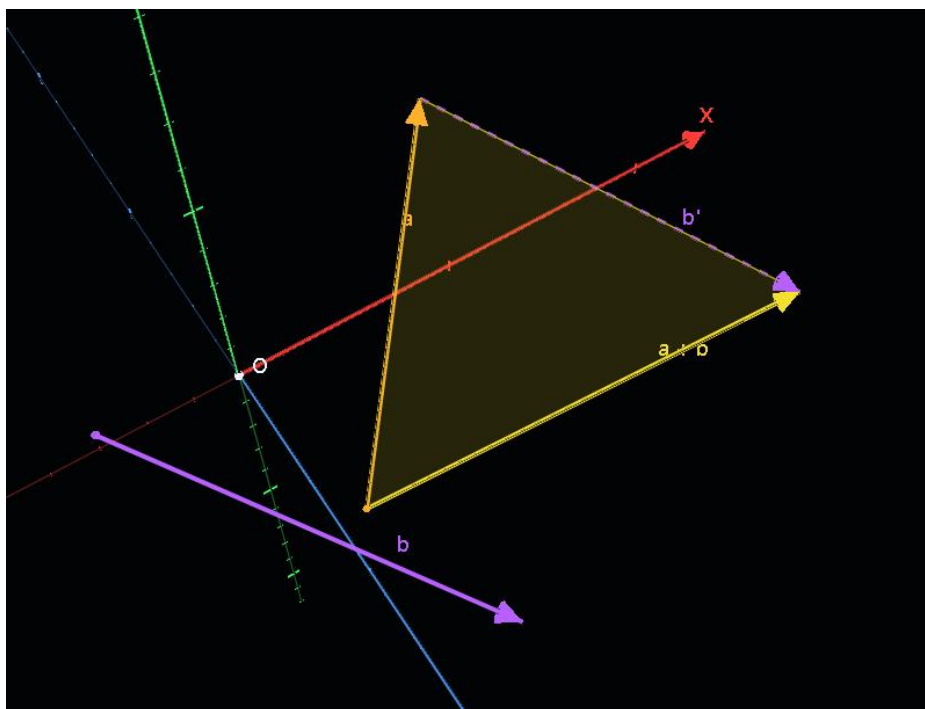


Рисунок 24. Геометричний результат суми вектора за правилом трикутника.

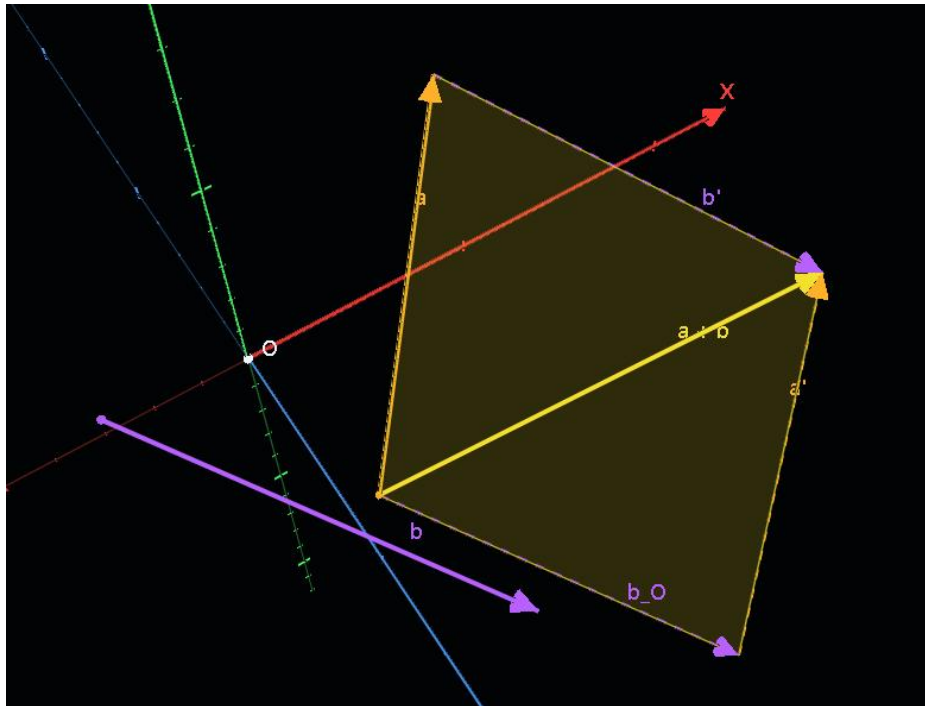


Рисунок 25. Геометричний результат суми вектора за правилом паралелограма.

Перевірка обчислення різниці векторів.

Скористуємося таблицею:

Вектор	Початкова точка	Кінцева точка	Отриманий вектор
$\vec{a}$	(-4; 6; -2)	(3; -1; 5)	(7; -7; 7)
$\vec{b}$	(2; -5; 4)	(-6; 3; -1)	(-8; 8; -5)

Таблиця 8. Вхідні дані для перевірки різниці векторів.

Різниця векторів дорівнює:

$$\vec{a} + (-\vec{b}) = (a_x - b_x; a_y - b_y; a_z - b_z) = (7 - (-8); -7 - 8; 7 - (-5)) = (15; -15; 12)$$

В результаті програма обчислила такий результатом:

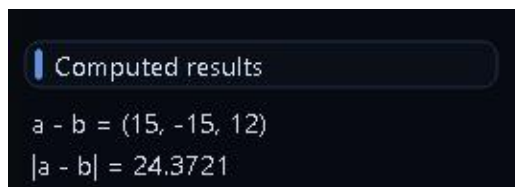


Рисунок 26. Числовий результат різниці векторів.

Програма показала даний результат, що задовольняє обчисленню. Тепер розглянемо, чи працює правильно операція геометрично.

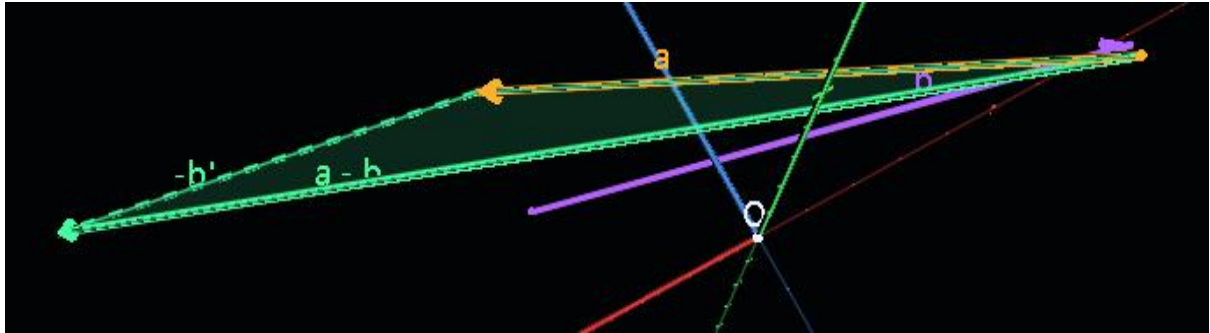


Рисунок 27. Геометричний результат різниці векторів.

Перевірка обчислення множення вектора на скаляр.

Для перевірки множення на скаляр перевірялося значення вектора і його довільний коефіцієнт для вектора. Скористуємося таблицею:

Вектор	Початкова точка	Кінцева точка	Отриманий вектор
$\vec{c}$	(5; -3; 8)	(-2; 4; -6)	(-7; 7; 14)

Таблиця 9. Вхідні дані для перевірки множення на скаляр вектора.

Нехай $k = -3$, тоді результат дорівнює:

$$\begin{aligned} \vec{k}c &= (k \cdot c_x; k \cdot c_y; k \cdot c_z) = ((-3) \cdot (-7); (-3) \cdot 7; (-3) \cdot (-14)) \\ &= (21; -21; 42) \end{aligned}$$

В результаті програма обчислила такий результат:

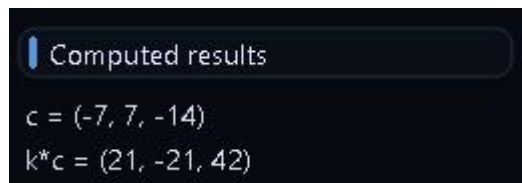


Рисунок 28. Числовий результат множення вектора на скаляр.

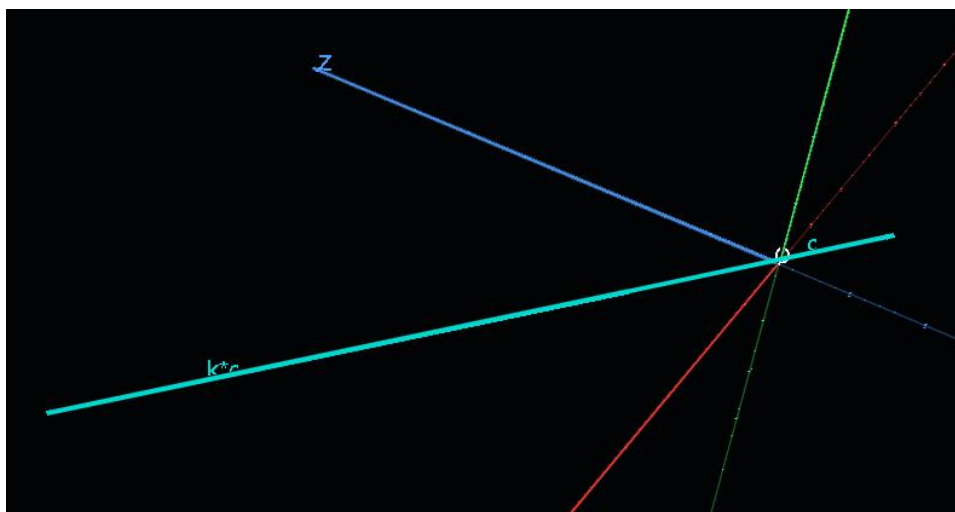


Рисунок 29. Геометричний результат.

Перевірка колінеарності векторів.

Скористуємося таблицею:

Вектор	Початкова точка	Кінцева точка	Отриманий вектор
$\vec{a}$	(-2; 1; -3)	(1; -1; 5)	(3; -2; -2)
$\vec{b}$	(4; -3; 1)	(-2; 1; -3)	(-6; 4; -4)

Таблиця 10. Вхідні дані для перевірки колінеарності векторів.

Умова колінеарності векторів:

$$\vec{a} \parallel \vec{b} = \frac{a_x}{b_x} = \frac{a_y}{b_y} = \frac{a_z}{b_z} \Rightarrow \frac{3}{-6} = \frac{-2}{4} = \frac{-2}{-4} \Rightarrow -\frac{1}{2} = -\frac{1}{2} = -\frac{1}{2}$$

Якщо їхні значення однакові та більше нуля, та їх довжина рівна нулю, тоді вони колінеарні з певним напрямком. В даному випадку: колінеарні і протилежний напрям.

В результаті програма такий результатом:

```

Computed results
a x b = (0, 0, 0)
|a x b| = 0
Result: collinear, opposite direction.

```

Рисунок 30. Числовий результат колінеарності векторів та визначення типу колінеарності.

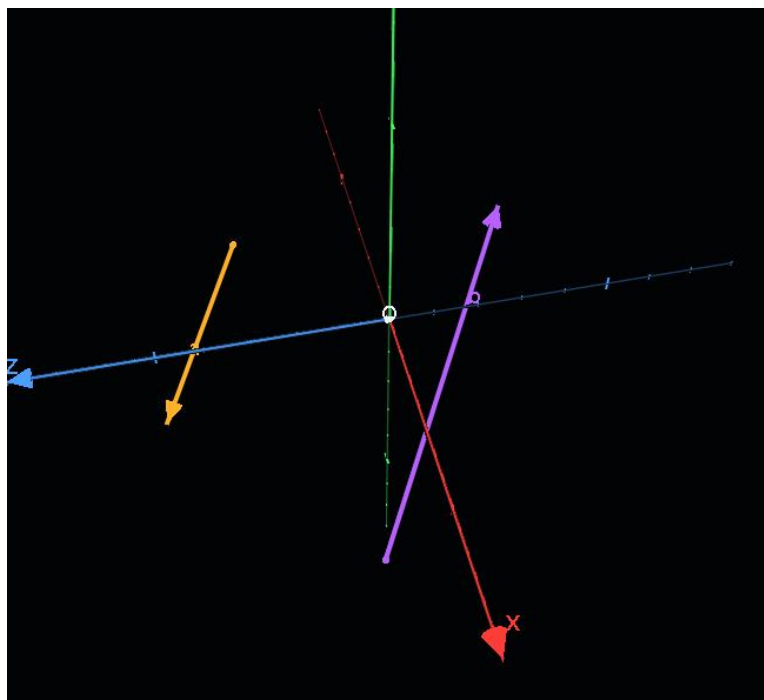


Рисунок 31. Геометричний результат.

Перевірка обчислення кута між векторами.

В даній перевірці використовувалися, ті ж координати, що в *таблиці 10*.

Обчислення кута між векторами:

$$\cos \alpha = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| \cdot |\vec{b}|} = \frac{-34}{\sqrt{17} \cdot \sqrt{68}} = \frac{-34}{\sqrt{17} \cdot 2\sqrt{17}} = \frac{-34}{34} = -1 \Rightarrow \alpha = 180^\circ$$

В результаті програма видає такий результат:

```
a * b = -34  
theta = 180 deg  
b is shown as a parallel copy from a's  
start point.
```

Рисунок 32. Числовий результат операції кута між векторами.

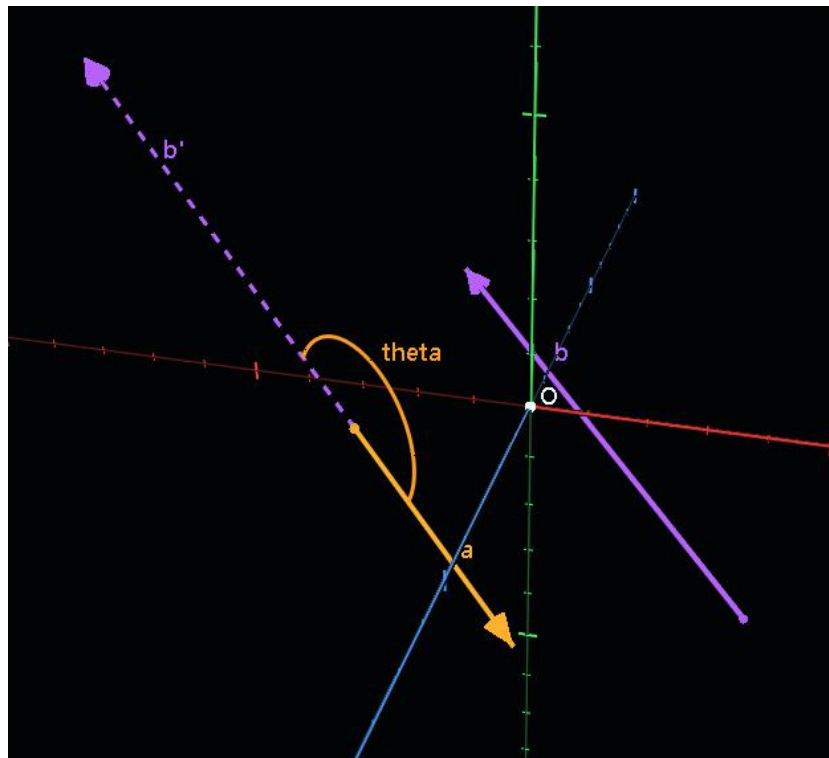


Рисунок 33. Геометричний результат.

Перевірка обчислення скалярного добутку.

Оскільки скалярний добуток використовується вже в визначенні кута між векторами, то в даній перевірці використовувалися, ті ж координати, що в *таблиці 10*.

Обчислення скалярного добутку дорівнює:

$$\vec{a} \cdot \vec{b} = (a_x \cdot b_x + a_y \cdot b_y + a_z \cdot b_z) = (3 \cdot (-6) + (-2) \cdot 4 + 2 \cdot (-4)) = -34$$

В результаті програма вивела такий результат:


```

Computed results
a * b = -34
a * b = 3*-6 + -2*4 + 2*-4
Angle type: obtuse.

```

Рисунок 34. Числовий результат скалярного добутку та визначення кута.

Перевірка ортогональності.

Скористуємося таблицею:

Вектор	Початкова точка	Кінцева точка	Отриманий вектор
$\vec{a}$	(-1; 2; 0)	(2; 0; 1)	(3; -2; 1)
$\vec{b}$	(1; -3; 2)	(2; 1; 3)	(1; 2; 1)

Таблиця 11. Вхідні дані для перевірки ортогональності векторів.

Обчислення ортогональності виконується за допомогою скалярного добутку, тому ортогональність дорівнює:

$$\vec{a} \cdot \vec{b} = (a_x \cdot b_x + a_y \cdot b_y + a_z \cdot b_z) = (3 \cdot 1 + (-2) \cdot 2 + 1 \cdot 1) = 3 - 4 + 1 = 0$$

В результаті програма вивела такий результат:

```

Computed results
a * b = 0
Result: orthogonal vectors.
Angle = 90 deg.

```

Рисунок 35. Числовий результат ортогональності.

Перевірка обчислення проєкції вектора на заданий напрямок.

Скористуємося таблицею:

Вектор	Початкова точка	Кінцева точка	Отриманий вектор
$\vec{a}$	(-2; 3; -1)	(4; -1; 5)	(6; -4; 6)
$\vec{b}$	(1; -2; 4)	(5; 1; 2)	(4; 3; -2)

Таблиця 12. Вхідні дані для перевірки ортогональності векторів.

Визначення проєкції: $proj_{\vec{b}} \vec{a} = \frac{\vec{a} \cdot \vec{b}}{|\vec{b}|} = \frac{0}{\sqrt{29}} = 0$

В результаті програма виводить такий результат:

```

Computed results
pr_b(a) = 0
proj_b(a) = (0, 0, 0)
a - proj_b(a) = (6, -4, 6)

```

Рисунок 36. Числовий результат проєкції.

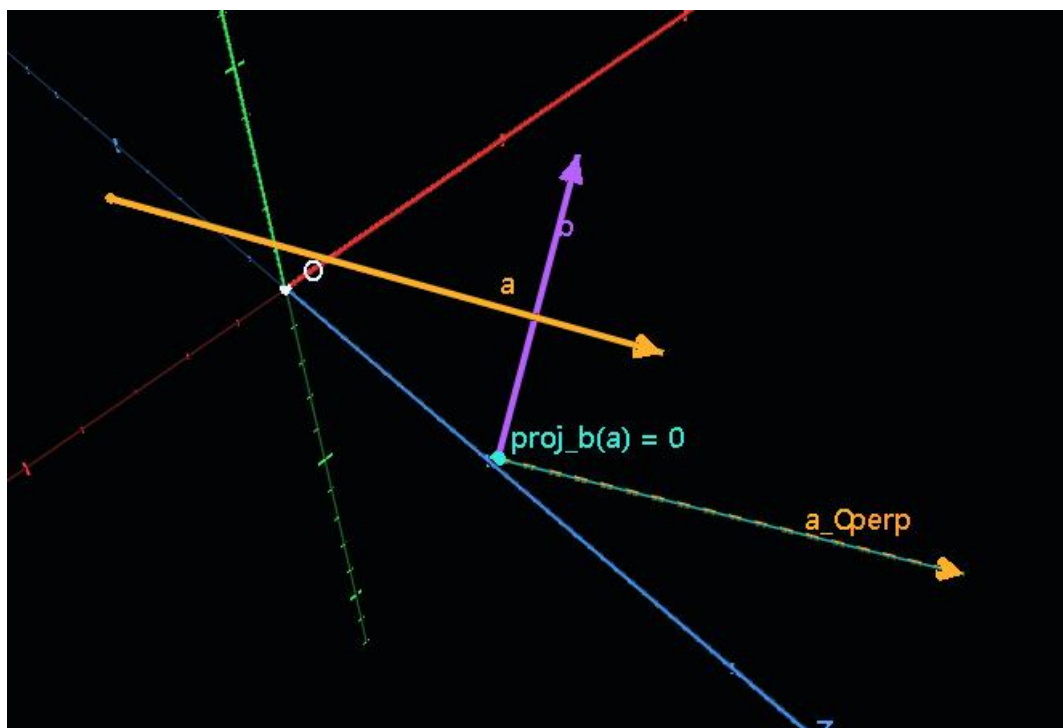


Рисунок 37. Геометричний результат.

Перевірка обчислення векторного добутку.

Скористуємося таблицею:

Вектор	Початкова точка	Кінцева точка	Отриманий вектор
$\vec{a}$	(1; -2; 3)	(5; 1; -1)	(4; 3; -4)
$\vec{b}$	(-2; 1; 4)	(1; -3; 2)	(3; -4; -2)

Таблиця 13. Вхідні дані для перевірки векторного добутку.

Обчислення даної операції дорівнює:

$$\vec{a} \times \vec{b} = (a_y b_z - a_z b_y) \vec{i} - (a_x b_z - a_z b_x) \vec{j} + (a_x b_y - a_y b_x) \vec{k} = (-22; -4; -25)$$

В результаті програма видає такий результат:

```
Computed results
a x b = (-22, -4, -25)
|a x b| = 33.541 (parallelogram area)
Right-hand rule: a x b sets the normal
direction.
```

Рисунок 38. Числовий результат векторного добутку та його довжини.

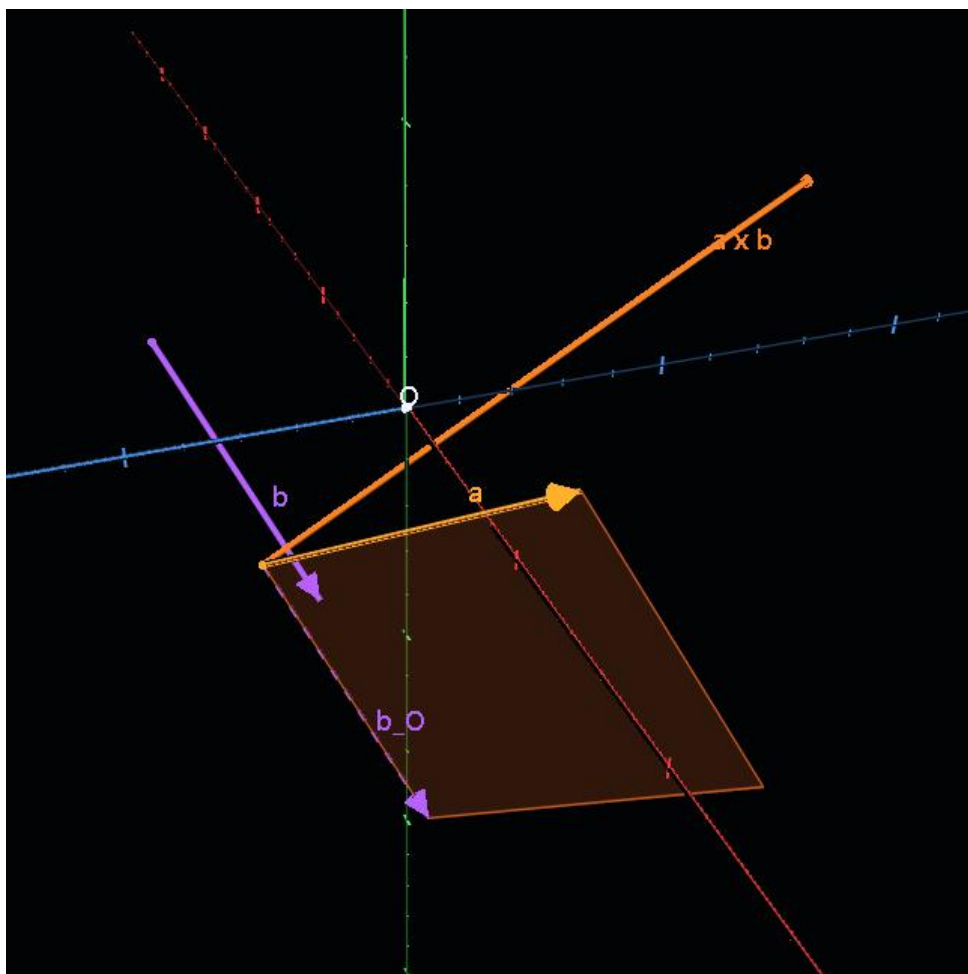


Рисунок 39. Геометричний результат.

Перевірка обчислення мішаного добутку.

Мішаний добуток є особливим, оскільки в ньому використовується вже три вектора. Скористуємося таблицею:

Вектор	Початкова точка	Кінцева точка	Отриманий вектор
$\vec{a}$	(1; -2; 3)	(4; 0; -1)	(3; 2; -4)
$\vec{b}$	(-1; 3; 2)	(2; -1; 5)	(3; -4; 3)
$\vec{c}$	(0; -3; 1)	(2; 1; 4)	(2; 4; 3)

Таблиця 14. Вхідні дані для перевірки мішаного добутку.

Обчислення даної операції дорівнює:

$$(\vec{a}\vec{b}\vec{c}) = \begin{vmatrix} a_x & a_y & a_z \\ b_x & b_y & b_z \\ c_x & c_y & c_z \end{vmatrix} = \begin{vmatrix} 3 & 2 & -4 \\ 3 & -4 & 2 \\ 2 & 4 & 3 \end{vmatrix} = -158$$

В результаті програма вивела такий результат:

```

Computed results
Uses vectors a, b and c.
b x c = (-24, -3, 20)
[a,b,c] = -158
Volume = |[a,b,c]| = 158

```

Рисунок 40. Числовий результат векторного добутку та його довжини.

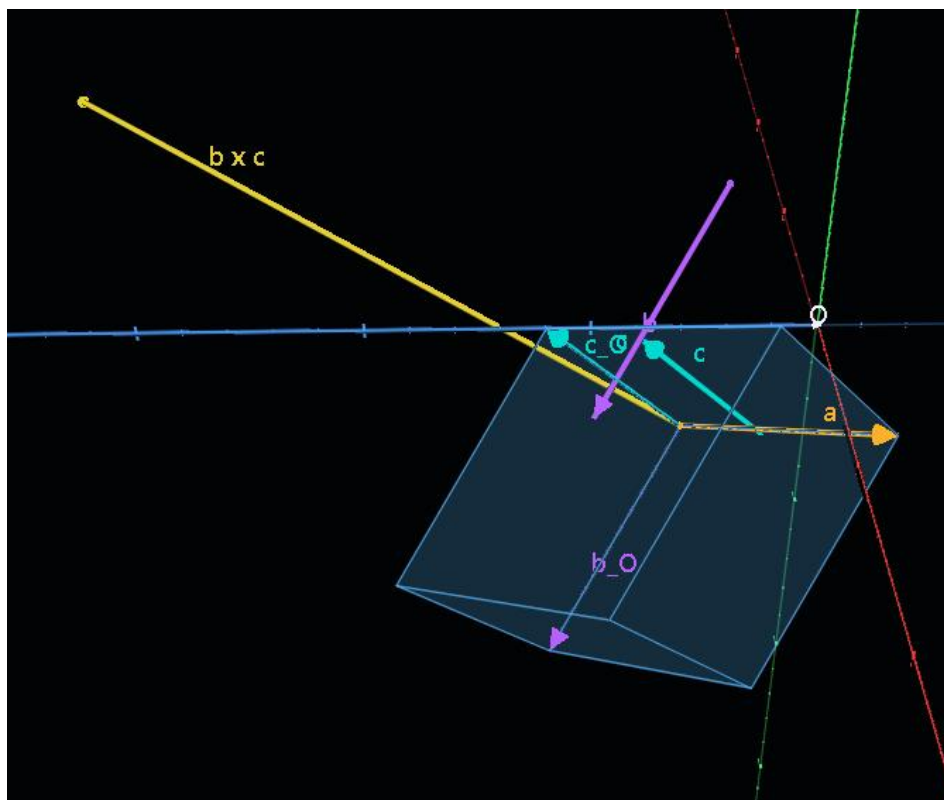


Рисунок 41. Геометричний результат.

Перевірка обчислення нормалі до площини.

Для даної операції використовуються такі ж координати, що в *таблиці 14*.

Обчислення площини має такий вигляд як і векторний добуток:

$$\vec{n} = \vec{a} \times \vec{b} = (-10; -21; -18)$$

В результаті програма обчислила такий результат:

```
Computed results
a x b = (-10, -21, -18)
|a x b| = 29.4109
N = (-0.34, -0.71, -0.61)
Right-hand rule: a x b defines N.
Enter vector c to show its projection on
this surface.
```

Рисунок 42. Числовий результат площини.

Особливість даної функції, що система повідомляє користувача ввести дані вектора $\vec{c}$ проєкції на площину, але навіть через відсутність, операція працює. Тепер після введення координат вектора $\vec{c}$ побачимо наступний результат:

```
Computed results
a x b = (-10, -21, -18)
|a x b| = 29.4109
N = (-0.34, -0.71, -0.61)
Right-hand rule: a x b defines N.
Projection source: vector c = (3, -4, -2)
Third vector c is projected onto the
surface plane.
proj_N(c) = (-1.04, -2.18, -1.87)
c_plane = (4.04, -1.82, -0.13)
```

Рисунок 43. Числовий результат площини з проєкцією.

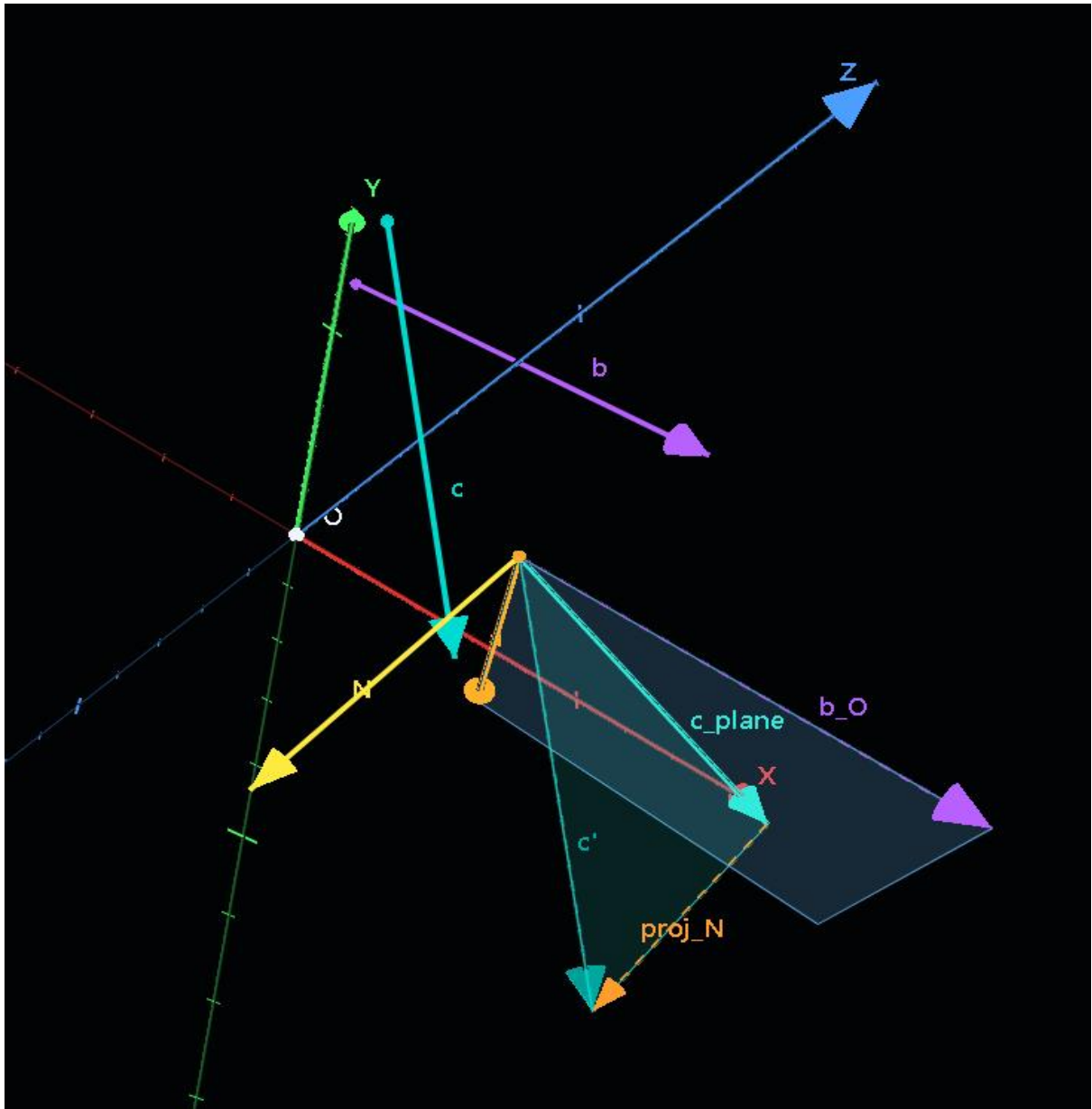


Рисунок 44. Геометричний результат.

Отже, ми бачимо, як перевірка програми з певними координатами, але також для кращої зрозумілості, дізнаємося в яких випадках дані перевірки спрацьовують для деяких операцій:

Операція	Вхідні дані	Очікувана поведінка програми
Кут між векторами	Один із векторів (0, 0, 0)	Повідомлення, що кут не визначено
Нормалізація	Вектор (0, 0, 0)	Операція не виконується як звичайна нормалізація
Проекція	Напрявлений вектор (0, 0, 0)	Повідомлення, що проекція не визначена

Векторний добуток	Паралельні вектори	Результат (0, 0, 0), нормаль не будується
Нормаль до площини	Два паралельні вектори	Повідомлення, що нормаль не визначена

Таблиця 15. Основні етапи побудови тривимірної сцени.

Такі перевірки потрібні для стабільної роботи програми. Якщо їх не врахувати, тоді програма могла би виконати ділення на нуль або побудувати некоректний графічний результат. Ось узагальнений результат перевірок:

Операція	Вхідні дані	Результат програми
Довжина вектора	8.77	Рисунок 22
Додавання	(9; 3; 2)	Рисунок 23, 24, 25
Віднімання	(15; -15; 12)	Рисунок 26, 27
Множення на скаляр	(21; -21; 42)	Рисунок 28, 29
Колінеарність	Колінеарні, протилежнонаправлені	Рисунок 30, 31
Кут між векторами	180 градусів	Рисунок 32, 33
Скалярний добуток	-34	Рисунок 34
Ортогональність	Ортогональні, значення рівне нулю	Рисунок 35
Проекція	0, вектор та складова однаково напрямлені	Рисунок 36, 37
Векторний добуток	(-22; -4; -25)	Рисунок 38, 39
Мішаний добуток	-158, об'єм фігури $ -158 = 158$	Рисунок 40, 41
Нормаль до площини	(10; -21; -18)	Рисунок 42, 43, 44

Таблиця 15. Узагальнений результат перевірок операцій векторної алгебри.

Таким чином, перевірка показала, що розроблена інтерактивний графічний застосунок коректно реалізує всі перераховані вище операції векторної алгебри. І для конкретної операції програма формує як числовий результат, так і графічний. Наявність демонстраційних прикладів і перевірка граничних випадків підтверджують математичну оптимізацію даного застосунку.

Висновки

У ході дипломної роботи було розглянуто застосування векторної алгебри в контексті сучасних методів комп'ютерної графіки та розроблено програмний застосунок для інтерактивної візуалізації основних операцій над векторами у тривимірному просторі

У першому розділі було подано алгебраїчну та геометричну інтерпретацію операцій додавання та віднімання векторів, множення вектора на скаляр, скалярного та векторного добутків, мішаного добутку, операцій нормалізації та проектування векторів. Здійснено огляд їх застосувань в комп'ютерній графіці для формалізованого опису координатних систем, напрямків, кутів, просторових перетворень, нормалей до площин, налаштування освітлення та керування камерою.

У другому розділі було описано процес створення програмного застосунку для наочного інтерактивного відображення векторів та результатів виконання операцій над ними, який включав реалізацію математичної моделі вектора та сцени, розробку модулів виконання обчислень, побудови тривимірного графічного об'єкта, керування камерою та обробки механічних компонентів для взаємодії з користувачем за допомогою клавіатури та миші. Для кожної векторної операції передбачено не тільки вивід результату у формі числових значень, а й графічного відображення у тривимірному віртуальному просторі, що надає можливість користувачу краще зрозуміти зміст цієї операції.

Також було виконано перевірку коректності роботи програми на контрольних прикладах, результати яких збігаються з очікуваними значеннями, а часткові особливі випадки обробляються окремо. Це дозволяє зробити висновок, що дана програма правильно оптимізована з урахуванням вимог математичної коректності та ефективності програмної реалізації.

Список використаних джерел

- [1] Собкович Р.І. Конспекти лекцій з аналітичної геометрії (у двох частинах). Частина 1. – Івано-Франківськ: Голіней О.М., 2016. – 236с.
- [2] 3D Math Primer for Graphics and Game Development. [Електронний ресурс].– Режим доступу: <https://gamemath.com/book/vectors.html>
- [3] OnlineMSchool[Електронний ресурс].– Режим доступу: <https://ua.onlinemschool.com/math/library/vector/>
- [4] Studfile[Електронний ресурс]. – Режим доступу: <https://studfile.net/preview/10214054/page:4/>
- [5] М.Ф. Пічугін, І.О. Канкін, В.В. Воротніков. Комп'ютерна графіка. Навчальний посібник: «Центр учбової літератури», 2013. – 346с.
- [6] Інна Глушак. Вступ до програмування мовою C++ – Івано-Франківськ: Голіней О.В., 2025. – 200с.
- [7] Іляш Ю.Ю. Навчальний посібник. Програмування (C++, ООП) ч.2 – Івано-Франківськ: 2023, Голіней. – 132 с.
- [8] Joey de Vries. Learn OpenGL - Graphics Programming– Kendall&Welling, June 2020. – 523с.
- [9] M.Pauline Baker. Computer Graphics With Opengl. Pearson; 4th edition (November 9, 2010). - 888p.